

國立成功大學
電機工程研究所
碩士論文

針對通用圖形處理器的智能熱管理方法

**An Intelligent Thermal Control Method for General-Purpose
Graphic Processing Units**

研究生：張峻豪

Student: Chun-How Chang

指導教授：邱瀝毅 博士

Advisor: Lih-Yih Chiou, Ph.D.

**Department of Electrical Engineering
National Cheng Kung University,
Tainan, Taiwan, R.O.C
Thesis for Master of Science**

July, 2020

中華民國一百零九年七月

An Intelligent Thermal Control Method for General-Purpose Graphic Processing Units

By

Chun-How Chang

A thesis submitted to the graduate division in partial
fulfillment of the requirement for the degree of
Master of Science

at

National Cheng Kung University
Tainan, Taiwan, Republic of China
July 2020

Approved by :

<u>Lilylin Chion</u>	<u>Ming Hwa Shan</u>
<u>C. H. Chen</u>	<u>Chia-Hsiang</u>

Advisor :

Lilylin Chion

Chairman :

Kang-Jen Yang

國立成功大學

碩士論文

針對通用圖型處理器的智能熱管理方法
An Intelligent Thermal Control Method for
General-Purpose Graphics Processing Units

研究生：張峻豪

本論文業經審查及口試合格特此證明

論文考試委員：

邱滙毅 許明華
陳中凱 郭致宏

指導教授：邱滙毅

系(所)主管：楊宏澤

中華民國 109 年 7 月 23 日

針對通用圖形處理器的智能熱管理方法

研究生：張峻豪 指導教授：邱瀝毅 博士

國立成功大學電機工程研究所 VLSI/CAD 組

摘 要

隨著物聯網(Internet of Thing, IoT)的成熟與 5G 通訊等技術的成長，邊緣運算(Edge computing)的概念逐漸受到重視，許多相關應用如機械手臂、自駕車、擴增實境(AR, augmented reality)等等也開始受到關注，由於這些應用特別強調即時的影像分析及影像辨識處理的能力，因此通用計算圖形處理器開始被廣泛的應用在這些工作上，然而大量的運算單元必然存在著相當高的功率消耗，在邊緣裝置朝著更小巧，功能更強大的應用發展趨勢下，高功率密度帶來的高溫效應逐漸受到關注，由於晶片開發早期難以獲取晶片功率與溫度等資訊，因而導致很高的熱複雜性被忽視的風險，使得許多熱議題在開發晚期才逐漸顯露。在這樣的背景之下，勢必需要結合更強大精準的熱建模以及更有效的熱管理技術才能克服這樣的挑戰。

本論文提出了一種針對通用計算圖形處理器的智能熱管理方法，在這個方法中會搭配功率與熱建模來獲取及時的溫度資訊，並針對通用計算圖形處理器內的每顆核心進行調整，進而讓處理器在維持一定的效能下亦可運作在安全的溫度內，實驗結果顯示，本論文所提出的方法與一般熱管理的演算法相比，整體而言平均可以降低約 13.6%的峰值溫度，同時只付出約 1.1%的效能代價與 1.3%的能量消耗。

關鍵字：邊緣運算裝置、通用計算圖形處理器、動態電壓頻率調整、熱管理

An Intelligent Thermal Control Method for General-Purpose Graphic Processing Units

Chun-How Chang

Lih-Yih Chiou, Ph.D.

Department of Electrical Engineering

National Cheng Kung University,

Tainan, Taiwan, R.O.C

SUMMARY

With the growth of Internet of Thing (IoT), the concept of edge computing is gaining popularity. Because many related applications focus on image processing or image recognition, the general-purpose computing on graphics processing units (GPGPUs) nowadays are widely used on mobile and edge devices because of its strong computing capability. However, with the increasing power density on limited devices volume, thermal issues become critical. High power density may generate serious thermal effects and the heat issues of chips cannot be ignored. Therefore, when designing a GPGPU for mobiles and edge computing devices, thermal management has become one of the main issues.

The thesis proposes an intelligent thermal control method for GPGPUs with low power techniques and task schedule. The proposed scheme can detect the temperature of each single instruction multiple thread (SIMT) core and dynamically adjust the voltage and frequency of different clusters of cores to reduce the peak temperature during runtime. According to the experimental results, when compared with the baseline approach, the proposed design can reduce temperature by 13.6% with merely 1.1% performance overhead and 1.3% energy overhead on average.

Key words: Edge Computing Devices, GPGPU, Dynamic Voltage and Frequency Scaling (DVFS), Thermal Management

INTRODUCTION

With the growth of Internet of Thing (IoT), the concept of edge computing is gaining popularity. Edge computing is an emerging technology and grows rapidly in recent years, which makes its applications and market booming. Compared with traditional centralized computing, edge computing can pre-process at edge devices and move the computation away from data centers towards the edge of the network. Therefore, it can speed up the overall data processing speed. Along with the development of edge computing, applications like artificial intelligence (AI) and 5G grow rapidly. The growth of edge computing devices is also predictable.

The general-purpose computing on graphics processing units (GPGPUs) nowadays are widely used on mobiles and edge devices because of its strong computing capability. However, thermal issues on mobiles and edge devices are gaining critical owing to the increasing power density on limited devices volume. Since GPGPU has a large number of hardware units to accelerate, the corresponding high power consumption will cause severe thermal effects to influence the performance, power, and reliability of the chips. Therefore, designer must make trade-offs among performance, power consumption, and thermal management. How to balance thermal and performance at runtime without excessive computing power becomes a crucial problem.

PROPOSED DESIGN

This thesis proposes an intelligent thermal control method for GPGPUs with low power techniques and task schedules for mobiles or edge computing devices. The method called thermal balance methodology (TBM) considers the temperature difference between the hottest core and the coldest core for thermal management and uses the low-power technique like clock gating, dynamic voltage and frequency scaling (DVFS) to eliminate dynamic power consumption. According to the temperature of cores, TBM will divide the SM cores

into two parts: the cluster of hot cores and the cluster of cold cores. With the suitable task schedule and clock gating, the cluster of hot cores will cool down by reducing power consumption and the voltage and frequency of the cluster of cold cores will be increased by DVFS to improve performance. The goal of thermal management is to decrease peak temperature on the GPGPUs and reduce performance overhead at the same time without excessive energy consumption.

Based on the temperature difference between the hottest core and the coldest core, TBM has different states when the temperature difference meets the threshold. First, when TBM starts to work, the module called task dispatcher unit, which is in charge of the task schedule, will adjust the task loading of every SM core. The task dispatcher unit will stop sending tasks to the cluster of hot cores, which lightens the task loading on the cluster of hot cores. After finishing the rest of tasks in themselves, the temperature of hot cores will decrease because of clock gating, which will eliminate most dynamic power.

As the cluster of hot cores cools down, TBM will increase the voltage and frequency of the cluster of cold cores to improve overall performance. By using low-power technique on the different clusters, TBM can transfer the task loading from hot cores to cold cores, which not only reduces the peak temperature but also avoids excessive performance overhead. Finally, when the temperature difference is under the threshold, the task dispatcher unit will start to send tasks to the cluster of hot cores and TBM will adjust the voltage and frequency of the cluster of cold cores to make the whole SM cores on the GPGPU work in the same speed.

RESULTS AND DISCUSSION

All simulations were performed by GPGPU electronic system level platform provided by CASLAB. The GPGPU under study is based on HSAIL ISA with 16 SM cores by configuring CASLAB-GPUSIM. In the whole GPGPU, there are PLL (Phase-Locked Loops)

[1] to adjust the frequency of the SM Cores, a DC-DC converter [2] to adjust the voltage value, and a TPM (Thermal & Power Management unit) to affect the task schedule in task dispatcher unit and manage the selection of the voltage and frequency of each cluster.

According to the experimental results, when compared with the baseline approach, the proposed design can reduce temperature by 13.6% with 1.1% performance overhead and 1.3% energy overhead on average.

CONCLUSION

In this thesis, we propose an intelligent thermal control method for GPGPUs with low power techniques and task schedules for mobiles or edge computing devices. The proposed method can decrease peak temperature on the GPGPUs and reduce performance overhead at the same time without excessive energy consumption. According to the experimental results, when compared with the baseline approach, the scheme can reduce the temperature by 13.6% with only 1.1% performance overhead and 1.3% energy overhead on average.

致 謝

這篇論文能夠順利完成，首先要感謝我的指導教授 邱瀝毅老師，不管是在課堂上的教學或是研究與計畫上的指導，都讓我在專業知識與研究方法上受益良多。也十分感謝老師讓我參與了許多研討會，能夠有機會精進自己的口條與國際觀。每當研究遇到瓶頸或是報告的表達不夠完善時，老師都能夠耐心地聆聽，並適時地給我建議與改進的方向，讓我能夠在最後完成自己的研究。在碩士班修業期間，老師也教會了我許多做研究應有的觀念及態度，使我獲益良多。

另外要感謝實驗室的學長姊：良盈、易溥、品修、宗翰、維軒、忠翰、哲彬、俊興、健棠、晉維、柏丞，在研究與生活上給予我很多建議，感謝我的同窗：憲譚、昱宏、育緯、紀寬、唯宏，在學習的路上有你們互相激勵，十分感激，而實驗室的學弟妹們：錫峰、志展、聖凱、宗蕙、偉碩、Tenzing、晨哲、鎮中、中凱、心妤、琬筠、韻如，謝謝你們的幫助與努力，也讓實驗室增添了許多活力。

最後感謝我的家人們及修如，在我修業的這段時光讓我能無後顧之憂地完成學業，與你們一同分享這個喜悅，謝謝你們。

張峻豪 謹誌

中華民國一百零九年七月 台南府城

目錄

摘 要	i
致 謝	vi
目 錄	vii
表目錄	ix
圖目錄	x
第 1 章 緒論	1
1.1 研究背景	1
1.1.1 邊緣運算	1
1.1.2 通用計算圖形處理器	2
1.1.3 功率消耗對溫度之影響與相關熱管理	3
1.2 研究動機	4
1.3 研究貢獻	5
1.4 論文架構	5
第 2 章 相關研究背景	6
2.1 通用計算圖形處理器整體架構	6
2.2 功率消耗之來源介紹與動態熱管理技術簡介	8
第 3 章 相關文獻探討	12
3.1 通用計算圖形處理器之功率管理	12
3.1.1 關鍵功率路徑分析	12
3.2 通用計算圖形處理器之相關動態熱管理	15
3.2.1 應用於多核心系統之具溫度感知之工作排程技術	15
3.2.2 應用於移動裝置平台之動態熱預測與功率管理技術	17
3.2.3 應用於多核心系統之混合式動態熱管理技術	18

3.3 相關文獻總結	21
第 4 章 應用於通用計算圖形處理器的動態熱管理方法	23
4.1 問題描述	23
4.2 目標平台之環境與架構	24
4.2.1 平台介紹	24
4.2.2 多重串流處理器內部架構	26
4.2.3 功率模型	29
4.2.4 熱模擬引擎	31
4.3 動態熱管理機制	33
4.3.1 熱溫差平衡機制	33
4.3.2 考量工作負載之動態閾值機制	36
4.3.3 工作排程機制	41
4.3.4 動態熱管理之流程圖	43
第 5 章 實驗結果與分析	44
5.1 實驗環境設置	44
5.2 實驗結果	46
5.2.1 不加入閾值限制之溫度管理比較	47
5.2.2 加入閾值之溫度管理比較	50
5.3 額外硬體付出	54
第 6 章 結論和未來工作	55
6.1 結論	55
6.2 未來工作	56
參考文獻	57

表目錄

表 3-1 相關文獻總結	21
表 5-1 CASLAB-GPUSIM 之實驗環境.....	44
表 5-2 CASLAB-GPUSIM 平台之頻率電壓設定.....	44



圖目錄

圖 1-1 邊緣運算市場大小及預測[3]	1
圖 2-1 通用計算圖形處理器架構(以 CASLab-GPU[16][17]平台為例).....	7
圖 3-1 文獻之系統架構圖[4]	13
圖 3-2 文獻之內部串流處理器[4]	14
圖 3-3 文獻之整體功率比例圖[4]	14
圖 3-4 文獻所使用之工作搬移[5]	15
圖 3-5 文獻所使用之 DVFS[5]	16
圖 3-6 文獻之演算法流程圖[6]	17
圖 3-7 文獻之 Hybrid DTM 系統架構圖[7]	18
圖 3-8 文獻之決策熱平衡[7]	19
圖 3-9 文獻之 DVFS[7]	20
圖 4-1 HSAIL[8]	24
圖 4-2 CASLAB-GPUSIM 平台架構圖 [8]	25
圖 4-3 多重串流處理器架構	26
圖 4-4 非記憶體元件功率資訊取得流程圖	29
圖 4-5 非記憶體元件功率模型誤差	31
圖 4-6 使用於熱模擬引擎之 GPGPU 佈局	32
圖 4-7 使用於熱模擬引擎之 GPGPU 冷卻機制	32
圖 4-8 熱溫差機制(一)	34
圖 4-9 熱溫差機制(二)	35
圖 4-10 熱溫差機制(三)	35
圖 4-11 熱溫差閾值	37
圖 4-12 平衡熱溫差流程	37

圖 4-13 熱溫差閾值範例.....	37
圖 4-14 熱平衡機制範例(一).....	38
圖 4-15 熱溫差閾值範例(二).....	38
圖 4-16 熱溫差閾值範例(三).....	39
圖 4-17 熱溫差閾值範例(四).....	39
圖 4-18 熱溫差閾值.....	40
圖 4-19 動態熱溫差閾值.....	41
圖 4-20 對不同 wg 數量於 SM 內部影響.....	42
圖 4-21 動態熱管理演算法流程.....	43
圖 5-1 功率與溫度管理架構.....	45
圖 5-2 針對數據密集計算型之測試樣本之實驗結果比較圖.....	47
圖 5-3 針對記憶體存取型之測試樣本之實驗結果比較圖.....	48
圖 5-4 針對混合型之測試樣本之實驗結果比較圖.....	49
圖 5-5 記憶體存取型串接數據密集計算型之實驗結果比較圖.....	50
圖 5-6 記憶體存取型串接數據密集計算型測試樣本之直線圖.....	51
圖 5-7 數據密集計算型串接記憶體存取型之實驗結果比較圖.....	52
圖 5-8 數據密集計算型串接記憶體存取型之實驗結果比較圖.....	52
圖 5-9 模擬每組熱管理技術之峰值溫度比較.....	53

第 1 章 緒論

1.1 研究背景

1.1.1 邊緣運算

近年來邊緣運算(Edge Computing)的技術發展如火如荼，其相關的應用及市場也正蓬勃發展，如圖 1-1 所示[3]，邊緣運算基於分散式運算的環境，在邊緣運算中，數據會在邊緣端預先做部分的處理，相較傳統的集中式運算和雲端運算將數據集中處理與分析，邊緣運算不再需要將所有的數據傳輸到本地數據中心進行處理和分析，而是將原本完全由中心節點處理的大型數據加以分解，切割成更小與更容易管理的部份並分散到邊緣節點去處理。這種方法能減輕網絡和服務器上的負載，特別是在工業物聯網（IIoT）中具有很高的應用性。一方面可以加快整體的資料處理速度，另一方面可以降低資料傳送所需的延遲時間，進而降低資料傳輸的成本。隨著逐漸普及的 5G 通訊技術與越來越多人工智慧(Artificial Intelligent, AI)的物聯網應用發展下，可以預期邊緣運算裝置將持續成長。

Internet of Things (IoT) connected devices installed base worldwide from 2015 to 2025 (in billions)

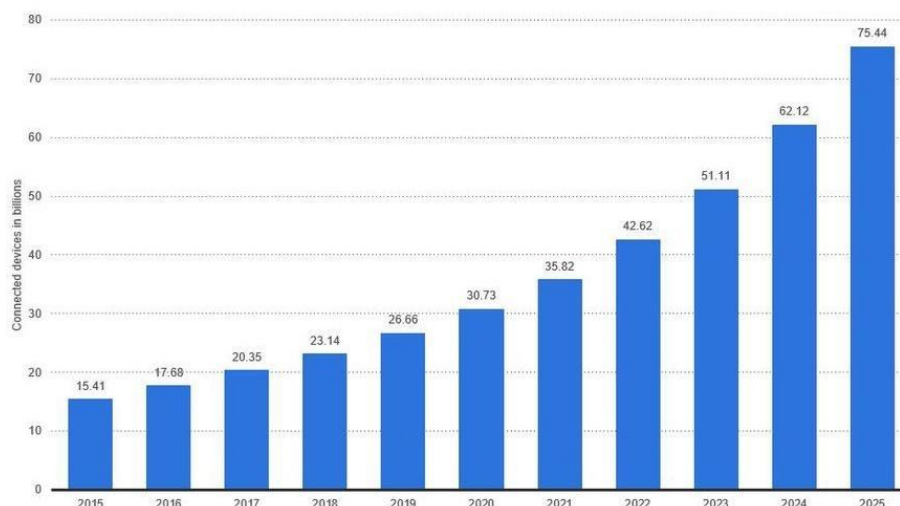


圖 1-1 邊緣運算市場大小及預測[3]

1.1.2 通用計算圖形處理器

隨著近年來人工智慧、影像辨識等研究領域越來越熱門，這其中最大的功臣莫過於通用計算圖形處理器 (General-Purpose computing on Graphics Processing Units, GPGPU) 晶片的發展。由於高度平行化的軟體編程方法以及具有大量硬體運算單元的優勢，使得圖形處理器漸漸能計算原本由中央處理器處理的通用計算任務而不再只是圖像式的相關工作，而這類型的通用計算圖形處理器相比中央處理器，其最大的優勢來自於處理單指令多資料流 (Single Instruction Multiple Data, SIMD) 的任務時可以有更強大的平行運算能力，因此在面對資料密集的非圖形應用程式資料時能夠展現其強大的並列處理能力和可程式化的流水線，除了應用於機器學習、影像辨識等領域外，像是 AI、VR/AR，或是自動駕駛技術的應用，雖然各有不同用途，但他們普遍都有一個共同的特色，都是需要大量平行運算 (Parallel Computing) 的能力，因此皆成為了通用計算圖形處理器廣為運用的領域。

而在通用計算圖形處理器高速運算的能力背後，除了靠著先進製成的進步外，其實大量的運算核心堆疊以及記憶體頻寬的擴展而來，這些高效率運算元件所產生的功耗以及其所引起的熱效應對晶片的負面影響已經來到不可忽視的地步，在開發早期的設計者缺乏這些相關資訊的情況下，要制定出一套合適的熱管理方法來應對這些潛在的高溫效應是一件相當大的考驗。

1.1.3 功率消耗對溫度之影響與相關熱管理

過高的功率消耗將會導致嚴重的熱效應，當晶片持續運作在高溫的區段時，對可靠性以及效能都會有所影響[9][10]，且漏電流也會與其所處的溫度有著正向的指數關係[11]，當溫度升高時，漏電流所造成的靜態功耗會大幅上升，而更大的功耗會導致溫度再次升高，形成高溫的惡性循環，因此如何在控管功耗上時同時考量溫度的效應，對熱管理來說是一個很大的挑戰，現今針對熱效應的管理方法，大致上可分為兩種：

考量溫度趨勢之晶片擺置方式[12]

當晶片持續運作且彼此間過於靠近時，容易產生熱耦合(Thermal Coupling)，所謂的熱耦合是指當兩塊發熱元件過於集中時，由於相較晶片分散擺置的方式來說缺少了一部份的散溫方向，因此相比之下較集中的兩塊發熱元件溫度會較高，如上述所言，晶片擺置的方式需要考量熱趨勢來設計，但是單純以晶片擺置並不能完全保證晶片在不同運作模式下都能保有良好的散熱性，因為封裝中每個運算元件的功耗都是根據當下的應用程式不同而有所不同，因此此種方法須同時考量運行中的硬體資訊。

動態熱管理方式[13][14][15]

動態熱管理機制是一種用於系統在運行中因避免過熱而調節的技術，其原理是利用在晶片運作的同時藉由觀測晶片的溫度變化來調控來解決潛在的熱效應，由於需要晶片的及時反饋，因此在開發早期需要仿真且準確的熱模擬模型來輔助，而根據熱管理控管的階層，大致上可以分成硬體動態熱管理與軟體動態熱管理，硬體動態熱管理常用到的技術如動態電壓與頻率調整(Dynamic Voltage and Frequency Scaling, DVFS)、時脈閘控(Clock Gating)、電源閘控(Power Gating)等等，由於這些異體動態熱管理直接影響功耗與頻率本身，相對於軟體端的動態熱管理而言，這些低功率技術能夠立即展現溫度下降的效果，但相對來說需要較多的能量耗損，而軟體動態熱管理技術通常實現於作業系統層級，像是工作搬移(Task migration)、工作排程(Task scheduling)等等，

藉由主動管理發放或是轉移工作的方式來達到控溫的效果，由於軟體端注重的是工作間的排程與指令的節流，軟體端的動態熱管理相較硬體而言較不會有大量的效能損失，但其控溫效應相對地也較為緩慢。

1.2 研究動機

如前言所提，邊緣運算的概念為許多應用帶來了全新契機。而在設計適用邊緣運算裝置的通用計算圖形處理器時，除了需要考量運算能力及功率消耗之間的平衡外，高功率密度所帶來的熱效應已然變成不可忽視的一環。考量到邊緣運算裝置一般都沒有配置強力的冷卻機制可以降溫，在無法預測運作的環境是否能夠自然降溫的情況下，勢必要考量到晶片能自主做熱管理來降低自身的溫度，避免過高溫度的產生。因此，如何適度使用熱管理的機制，同時不降低過多的運算能力成為了必須要被解決的問題。以往針對通用計算圖形處理器的熱管理方法通常只針對整體的操作電壓點與頻率來做優化，很少考量到通用計算圖形處理器內部核心個別運作時彼此牽連的熱效應，而若將實現在中央處理器上的溫度控管機制轉移到通用計算圖形處理器上，效果將會十分有限。主要是因為通用計算圖形處理器的運算能力來自於高平行化以及多核心的架構，使得通用計算圖形處理器在運行過程中的控制機制和資料流比以往的中央處理器複雜許多。這也造成熱管理上的困難，因此運作在通用計算圖形處理器上的熱管理機制勢必要考量到其本身的工作原理，才能在溫度與效能上達到雙贏的局面。

本論文利用通用計算圖形處理器運行時的硬體動態資訊與即時溫度，動態調整通用計算圖形處理器中不同區塊的開關與整體的電壓頻率，並且搭配合適的工作派發排程來提高降溫效率，以降低熱耦合的產生，並且確保晶片運作在可接受之安全範圍內，以降低對可靠性與使用壽命的損害。

1.3 研究貢獻

本論文提出一套適用於邊緣運算裝置中通用計算圖形處理器的智能動態熱管理方法，這套動態熱管理方法具有以下特點：

- ♦ 同時考量硬體的即時溫度回饋與硬體的運作情況，進而對不同的硬體區塊做熱管理
- ♦ 考量當系統超過最大溫度需要緊急停止時，能正確關閉運作中的處理器核心
- ♦ 動態溫度閾值調整，具有可調性，可以適應不同的應用情境

1.4 論文架構

本論文在第 1 章中介紹本篇論文的研究背景、研究動機以及研究貢獻。第 2 章講述本篇論文需要的背景知識，包含了通用計算圖形處理器內部的架構、動態熱管理介紹等等。第 3 章討論過去一些適用於圖形處理器或是異質架構平台上的動態熱管理研究，並對這些研究做出說明和整理。第 4 章，詳細說明本篇論文所提出的智能熱管理方法。第 5 章則是實驗環境的介紹以及實驗結果的說明和分析。第 6 章針對本篇論文做出結論以及探討未來可以繼續進行的目標和方向。

第 2 章 相關研究背景

在這個章節會講解一些相關的背景知識，在 2.1 節中會介紹通用計算圖形處理器的架構，在 2.2 節則是解說功率消耗的種類以及動態熱管理技術的簡介。

2.1 通用計算圖形處理器整體架構

圖形處理器(GPU)是一種專門執行繪圖運算工作的微處理器，不同於傳統的 CPU，GPU 具有大量的運算核心，這些核心可以並列執行大量計算來加速顯示與繪圖等相關的工作項目，由於現代圖形處理器有越來越強大的並列處理能力和可程式化的流水線，使圖形處理器漸漸能計算原本由中央處理器處理的通用計算任務而不再只是圖像式的相關工作，這種通用計算圖形處理器在面對資料密集的非圖形應用程式資料時能夠展現其高度平行化的軟體編程方法以及具有大量硬體運算單元的優勢，在效能上大超越了傳統的中央處理器應用程式。

通用計算圖形處理器的架構為單指令多資料流架構(Single Instruction Multiple Data, SIMD)，而其中的運算核心單元稱之為串流處理器(Streaming Processor, SP)，這些串流處理器集成成為多重串流處理器(Streaming Multiprocessor, SM)，可視為通用計算圖形處理器中的運算處理核心，而在多條執行緒(thread)的結合下形成了單一指令多線程(Single Instruction Multi Thread, SIMT)的運算模式。

CASLab-GPUSIM

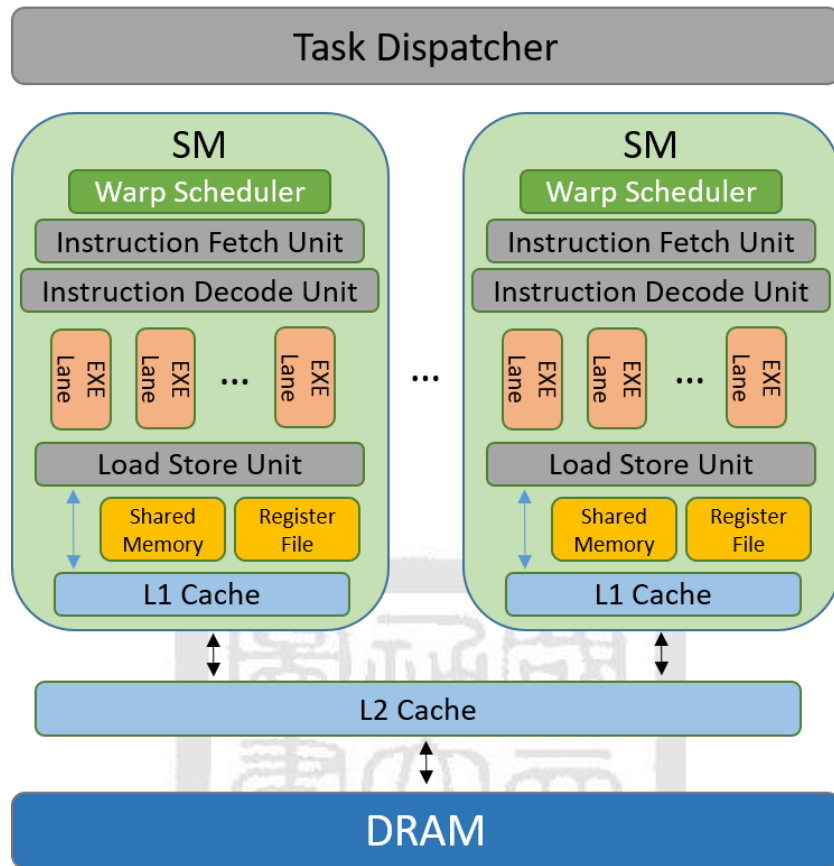


圖 2-1 通用計算圖形處理器架構(以 CASLab-GPU[16][17]平台為例)

在圖 2-1 當中可以看到通用計算圖形處理器的整體架構，主要由 Interconnection Network、SM(Streaming Multiprocessor)、Workgroup Scheduler(又稱 Task Dispatch Unit) 所組成，其中 Interconnection Network 負責連接外部 DRAM 和各個元件之間以及資料的搬運。SM 則是運算與執行的主要核心單元，最後 Workgroup Scheduler 是用來接收工作以及派發 Workgroup 的元件。當 CPU 派發工作時，會以 Grid 為單位接收所要執行的程式，在進行切割與排程後，會以 Workgroup 為單位派發至每個 SM 去做執行。而在 SM 在收到 Workgroup 後會根據 SIMD 的寬度分成多個 Warp，並以 Warp 為單位進行運算，同一個 Warp 內的執行緒會平行運算，而當工作進入 SM 內部後，Front-end 的 Warp Scheduler 會根據特定排程管理多個 Warp 並交由 Back-end 運算單元做處理，接著會根據指令類別分別到 Execution Unit 做平行運算或是到 Load Store Unit 等

待記憶體存取，藉此展現 Data-level Parallelism 或是 Memory-level Parallelism 的效益，可以觀察到 GPGPU 的資料流程有利於平行運算的特性，因此能在可平行化的應用程式之中得到大量的效能提升。而記憶體(Memory Partition)的部分可分為第二級快取記憶體(L2 Cache)以及動態隨機存取記憶體(DRAM)，此處是所有的 SM 共用記憶體區塊，藉由中介的 Interconnection Network 可以讓資料在 SM 與 L2 Cache、DRAM 間做交換。

2.2 功率消耗之來源介紹與動態熱管理技術簡介

由於溫度取決於功耗的組成，因此在介紹熱管理技術之前會先說明功率消耗的來源。後半段則會針對目前使用在動態熱管理的相關技術做介紹。

功率消耗的種類與來源

在 CMOS 電路之中，總功率消耗主要可以分成兩種類別，分別是：由電容充放電引起動態功耗(Dynamic Power)以及由漏電流引起的靜態功耗(Static Power)，總功率的公式如下圖 2-2:

$$P_{Total} = P_{Dynamic} + P_{Static} \quad (2.2)$$

對於 CMOS 電路來說，動態功耗可以分為開關功耗(Switching Power)和內部功耗(Internal Power)，兩者公式如圖 2-3、2-4，基本上動態功耗主要與三個參數有關，分別是節點電容、工作頻率、以及核心電壓。這些參數與動態功耗皆呈現正比例相關，因此控制頻率與電壓是一種有效控制動態功耗的方式之一。

$$P_{Switch} = \frac{1}{2} \times C_{load} \times V^2 \times f \quad (2.3)$$

$$P_{Internal} = \left(\frac{1}{2} \times C_{int} \times V^2 \times f \right) + (V \times I_{sc}) \quad (2.4)$$

當電晶體處於截止狀態會存在反向偏壓的漏電流，靜態功耗即是由漏電流引起的功耗，和動態功耗相比，靜態功耗取決於核心電壓與漏電流的乘積，特別注意的是由於二極體的漏電流會隨溫度上升而增加，因此當晶片持續運作在高溫的情況下，容易產生由靜態電流與溫度反覆回饋所導致的溫度竄升。相關的公式如下圖：

$$P_{Static} = V \times I_{leak} \quad (2.5)$$

如同前述介紹，總功率可以分為動態功耗以及靜態功耗，前者與頻率電壓有直接的關聯性，而後者則與電壓以及所在的溫度有關，因此熱管理如何遏止溫度的上升，其挑戰便在於如何使用低功耗技術來妥善分配功率、消散熱能、以及防止高溫熱點的產生。

動態熱管理技術簡介

近年來的動態熱管理技術大致可依照調控所處的層級分成硬體端動態熱管理與軟體端動態熱管理，硬體端的動態熱管理使用了在設計電路中一些常見的低功耗技術，而軟體端動態熱管理則是考量發配給各顆核心的工作分配機制，除了使用管理與控制等技術外，使用如風扇、水冷技術等的物理性降溫也可歸類於熱管理內的方法之一，以下內容則著重介紹在本篇論文中有提到的動態熱管理技術以及其所搭配的低功耗技術一併做介紹。

(1) 工作排程(Task Schedule)[15][18]

工作排程通常是指作用在作業系統層級的工作排程，能夠安排系統內各顆核心的工作類型，另外一種常見的是具溫度感知的工作排程(Temperature-aware Task Schedule)，能夠在決定排程的時候額外考量溫度的回饋，來達到核心溫度與效能的平衡。

(2) 工作搬移(Task Migration)[19]

工作搬移經常使用在多核心系統的軟體層級，藉由搬移不同核心間的工作來使峰值溫度(Peak Temperature)降低，反方面來說也能提高核心間的使用率，因此使用此類方法通常會提高平均溫度。

(3) 動態電壓與頻率調整(Dynamic Voltage and Frequency Scaling)[20]

DVFS 是一種電源管理的低功耗技術，能夠根據處理器不同的使用狀況來調整頻率與電壓，除了使用於功率管理外，當不同區塊根據晶片實際布局的相對位置與每顆核心間相異的工作負載，導致有熱溫差的產生時，熱管理機制可以在蒐集完系統當前資訊之後，對於溫度較高的核心給予較低的頻率以及電壓來降溫，而對溫度較低的核心則是升高頻率和電壓來增溫，最終避免熱溫差的分布。

(4) 時脈閘控(Clock Gating)[21]

是一種在同步序向邏輯電路(Synchronous Sequential Circuit)中的技術，時脈閘控會在時脈樹(Clock Tree)中增加一些邏輯閘，在不需要的時候將時脈關閉，在降低動態功率消耗的同時，也能減少溫度的上升。

(5) 電源閘控(Power Gating)[22]

電源閘控的優點是可以透過關閉工作區塊的電源而進一步省去靜態漏電流的功耗，達到降低靜態功耗的效果，但是設計上也需要付出不少代價，比如在關停電源的狀態下復電會需要考慮一段時間的延遲，另外在開關之間，電路沒有辦法接收忽高忽低的電壓值，會需要隔離器(Isolator)來做使用，造成多餘的面積代價。

(6) 散熱片與風扇(Heat sink & Fan)

利用散熱片與風扇來達到晶片的降溫已經是熱管理技術中最常見的方法，雖然近年來出現的各種不同冷卻技術如水冷、冷凍循環以及浸入式沸騰冷卻等都可能用來解決散熱問題，但是考量到成本與可靠性，散熱片還是晶片在物理性降溫上的首選。

(7) 其他：其餘低功耗技術包含多電源設計(Multi-Voltage)[23]、Data Gating、Memory Gating、SLEEP Transistor Technique、Emerging Memory 等。

綜合上述所提到的，動態熱管理的核心概念無非是為了降低熱點的產生，進而在高溫的核心上安插較少的工作負載，並使用低功耗技術來讓整體功耗降低，而為了最小化效能的損失，同時減低最高溫度的飆升，相反地在較低溫的核心上則安排較多的工作負載，並提升頻率與電壓，藉此平衡每顆核心的溫度差異，而設計動態熱管理的難點在於如何判斷當前運行過程中，是適當的熱節流時段，除了考量當前溫度的回饋外，也需要有硬體實際的運作狀況，並考量實作低功耗電路所需要多付出的代價等等，如此才能在效能(performance overhead)與熱效應間做取捨。



第 3 章 相關文獻探討

由於溫度升降是源自於元件上的功率變動，因此許多熱管理技術會同時考量功率管理的部分，在這個章節之中會探討適用於通用計算圖形處理器或是結合中央處理器的異質架構平台的動態熱管理與低功耗技術相關的文獻。在 3.1 節之中會先介紹在通用計算圖形處理器中的功率管理方法，在 3.2 節中則是討論在結合中央處理器的異質架構平台中的動態熱管理方法。

3.1 通用計算圖形處理器之功率管理

3.1.1 關鍵功率路徑分析

在文獻[4]這篇論文之中，作者提出了一種動態功率管理方法，是基於計算圖形處理器運行時的硬體動態資訊，將運行中的應用程式分為運算密集、記憶體存取密集或是趨勢不明顯的狀況，並藉由分析硬體動態資訊動態對通用計算圖形處理器中不同的區塊依據運行的狀況做開關或是電壓頻率的調整。整體的系統架構如下：

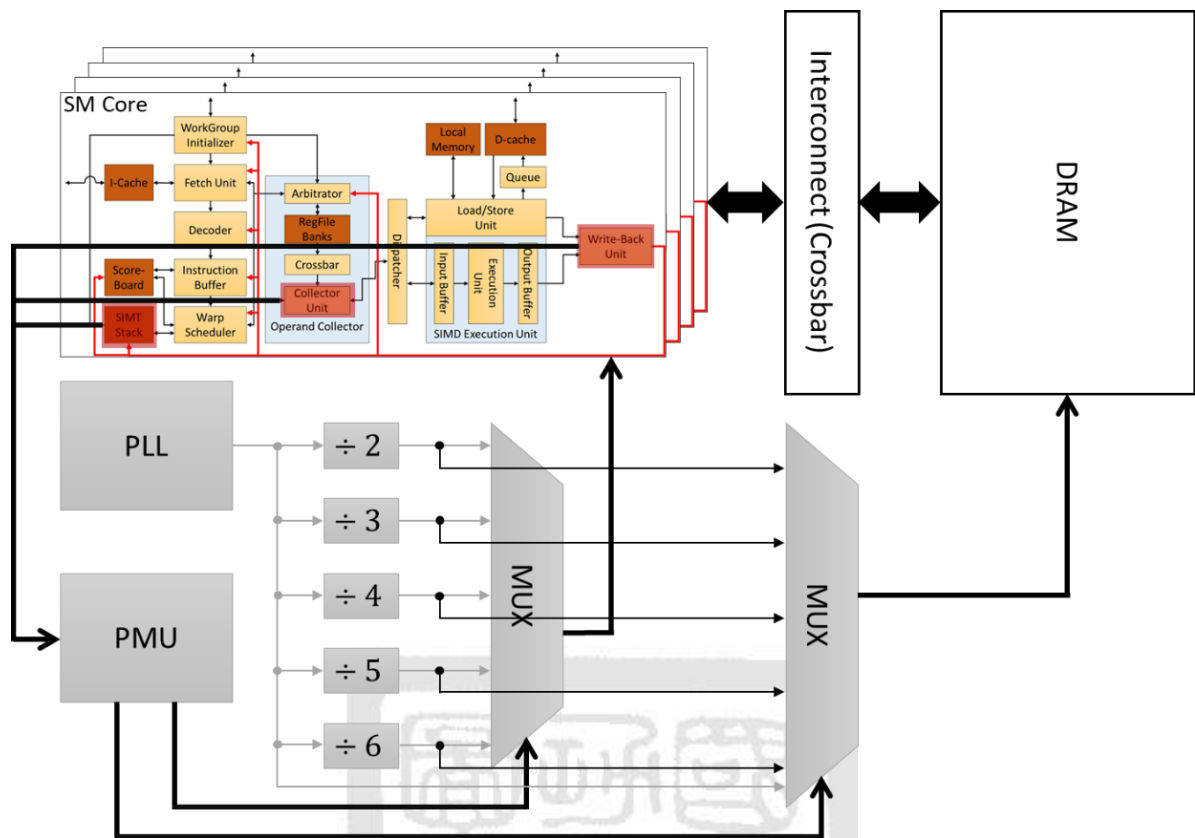


圖 3-1 文獻之系統架構圖[4]

這篇論文主要提出了一個在通用計算圖形處理器運行期間，動態偵測當前的運算核心使用率來進行不同區塊的動態電壓頻率調整，進而達到高能量效率的目標，而在判別何為當前運行之中效能關鍵的區塊中，作者針對當前的平台建立一套功率模型 (Power Model)，此功率模型基於 TSMC 40nm 製程所產生的硬體資訊作功耗分析，能夠在應用程式運行於通用計算圖形處理器平台時依據不同的行為進行功率的計算。

作者利用功率模型找出運行期間功率最大的三種元件區塊，並以此三種元件的使用率作為當前硬體運作的指標，分析當前應用程式是運算核心使用密集(Core-intensive)或是記憶體存取密集(Memory-intensive)的趨勢，若是運算核心使用密集，則表示目前功耗的關鍵路徑在核心的位置，因此調升核心的電壓頻率並降低記憶體的電壓頻率，反之若為記憶體存取密集，則降低核心的電壓頻率並調升記憶體的電壓頻率，讓關鍵功率路徑上的原件能得到最大的資源。

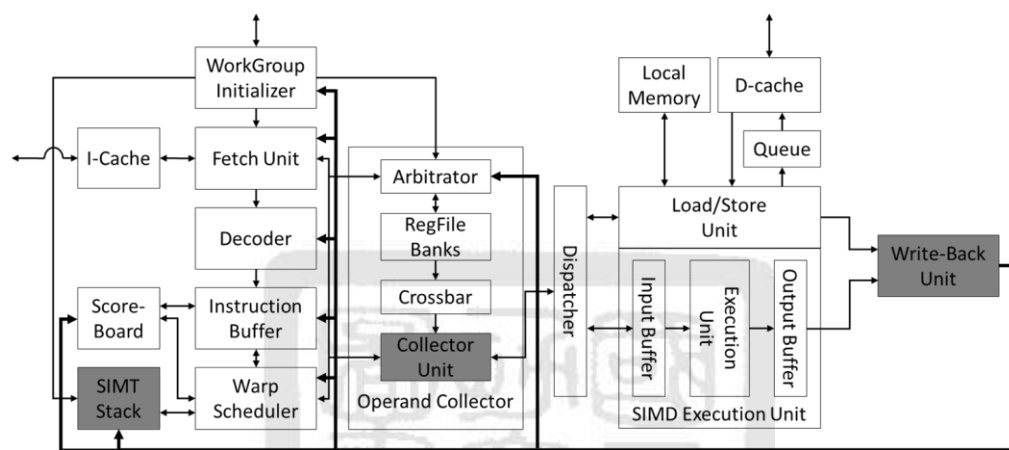


圖 3-2 文獻之內部串流處理器[4]

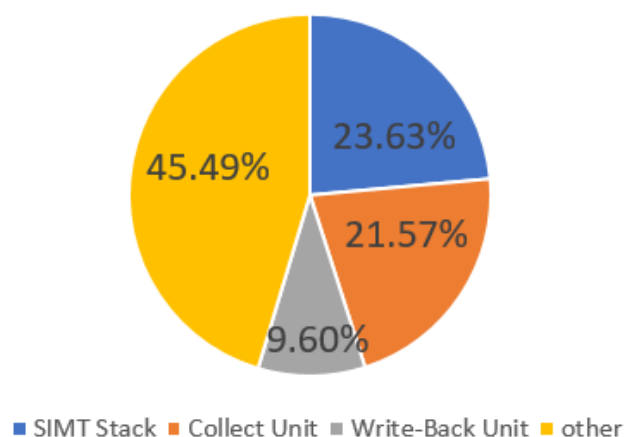


圖 3-3 文獻之整體功率比例圖[4]

3.2 通用計算圖形處理器之相關動態熱管理

由於動態熱管理技術較常使用於多核心系統平台或是實現在結合 CPU-GPU 的異構計算平台，因此在這個章節中我們會一併介紹在不同平台上的動態熱管理相關概念

3.2.1 應用於多核心系統之具溫度感知之工作排程技術

在相關文獻[5](ICMSAO'19)中提到了一種動態熱管理技術，主要¹是應用於多核心 CPU 平台上，隨著單位體積的電晶體個數增加，多核系統的功率密度與溫度也隨之提高，因此該論文提到了一種多核系統的運作模式稱為 dark silicon era，在該模式下所有平台上的核心不會同時運作，此時會以仍在運作的核心與關閉的核心為區別，沒有運作或是處於關閉的核心稱為 dark core，為了不讓核心因為彼此相鄰而產生熱耦合，會將有運作的核心與關閉的核心相互交錯的擺置。該論文也提出了一種任務調度技術，如圖所示，通過將任務從正在活動的內核轉移到 dark core 來提高系統性能並平衡內核之間的熱量分配。

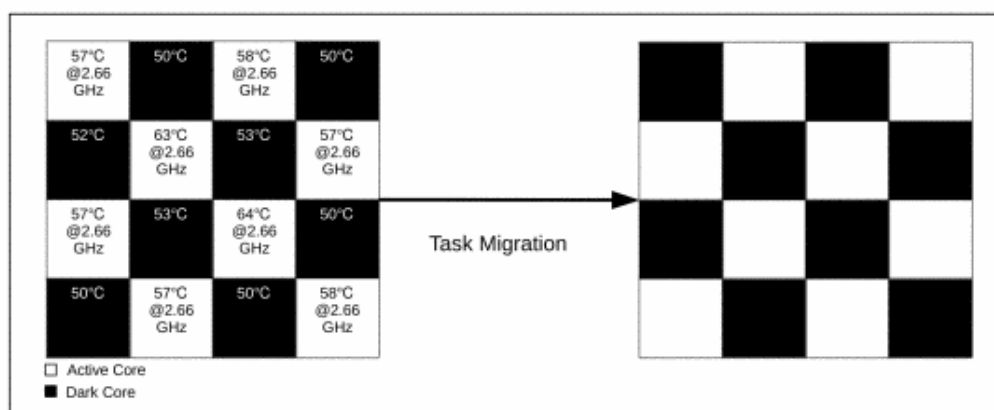


圖 3-4 文獻所使用之工作搬移[5]

另外針對即時的溫度回饋，該論文使用了動態電壓頻率縮放（DVFS）來針對每顆核心做頻率與電壓的調整，如圖右所示，在違反該論文提出的熱限制的情況下會降低正在活動的核心速度，以此將系統溫度保持在安全的工作範圍內。

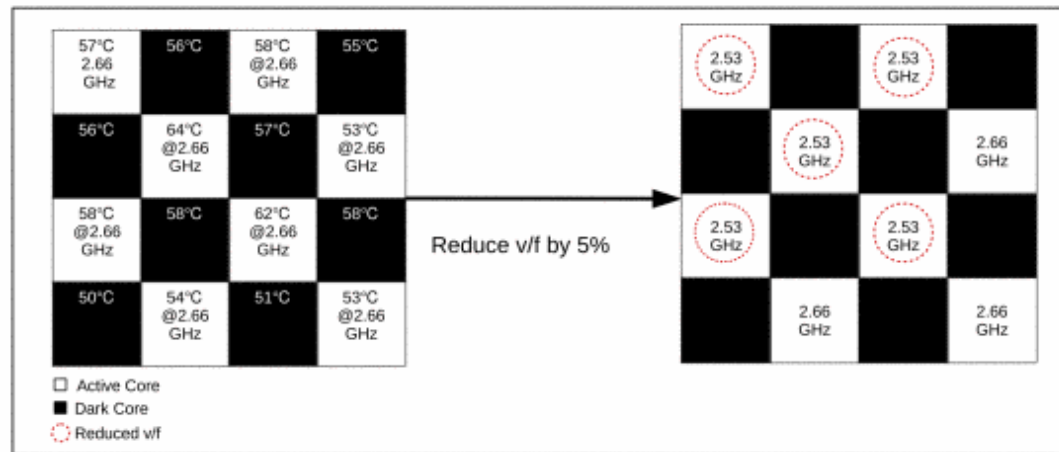


圖 3-5 文獻所使用之 DVFS[5]

3.2.2 應用於移動裝置平台之動態熱預測與功率管理技術

文獻[6]是使用了在異質多處理器架構平台的動態熱和功率管理 (DTPM) 算法，此篇論文的平台為三星 Exynos 5410 芯片，該芯片具有四個 A15 的高性能核心群集與四個 A7 的節能核心群集，並配有圖形處理器，由於在智能手機等移動平台上架設冷卻裝置如風扇並不是一種可行的行為，因此需要靠系統內部的演算法來計算出合適的運行功率來避免熱議題。該論文提出了一種預測方法，可以利用核心的使用率與其所建立的功率模型來推算下一刻的溫度趨勢，整體演算法流程如圖所示：

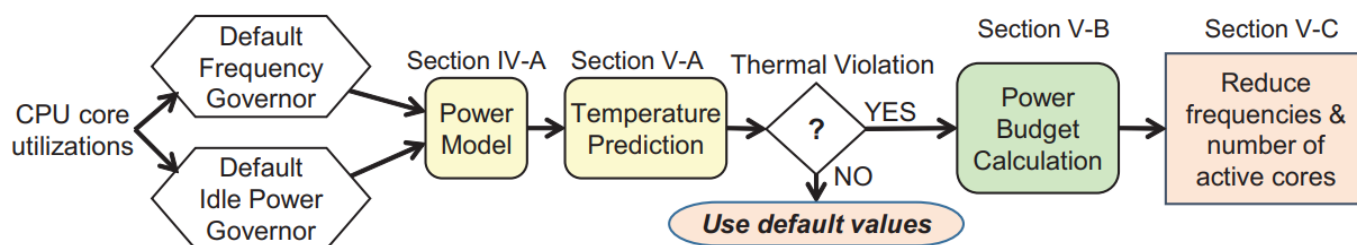


圖 3-6 文獻之演算法流程圖[6]

在無演算法的情況下，移動平台的作業系統會使用較多的核心來運作並逐漸升高頻率來增快處理效率，並未考慮到過熱的可能，因此本篇論文利用演算法推估未來的溫度並按照回饋調整頻率電壓，當預測的溫度結果違反熱閾值時，藉由計算所需的功率來調整頻率與關閉核心的數量，藉此避開上升溫度導致的熱效應。另外此篇論文中針對多核心的 CPU 系統是採取 DVFS 個別調整每顆核心的頻率電壓，且可以調整核心的運作數量，對 GPU 則是直接使用全體降頻與升頻的動作，並沒有對 GPU 上的每顆串流處理器個別調整，可以看的出針對圖形處理器上的熱管理方式較少會去考量每顆核心間的差別。

3.2.3 應用於多核心系統之混合式動態熱管理技術

文獻[7]提出了一種運作在 3D IC 封裝多核心系統的混合式動態熱管理方法 (Hybrid DTM)，所謂混合式動態管理方法，是指結合在作業系統層級使用工作搬移的軟體熱管理與考量溫度效應的硬體熱管理方法，其系統架構如下：

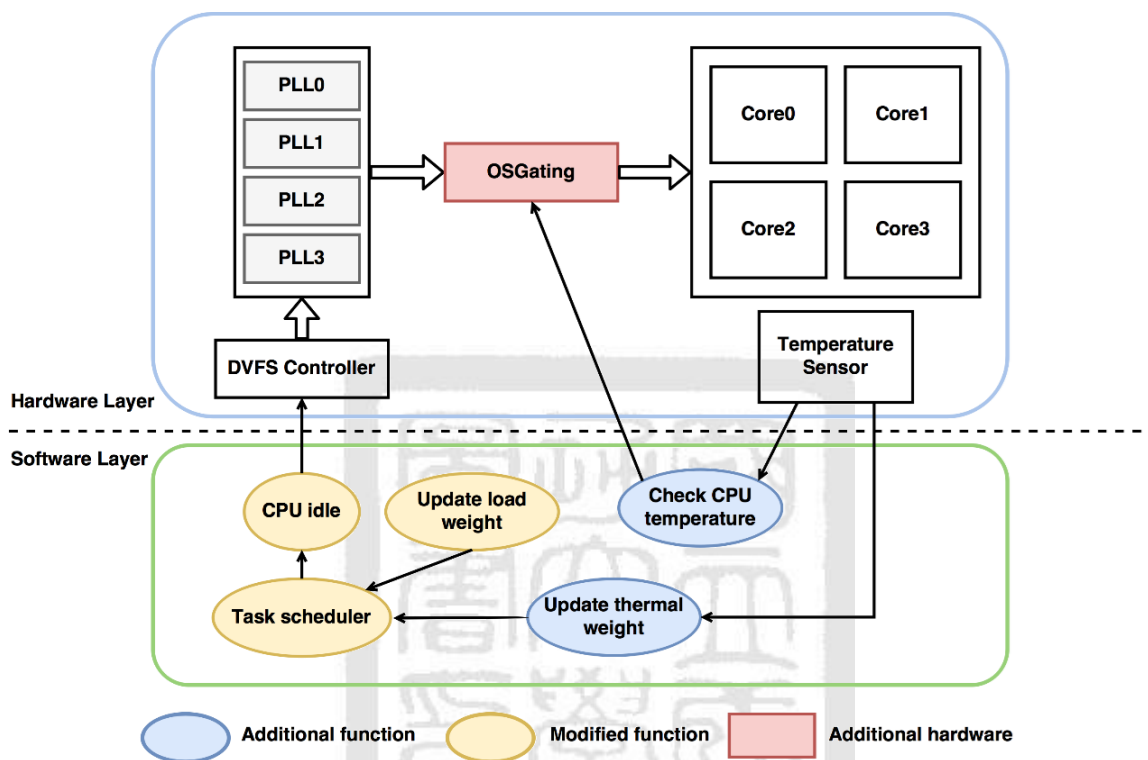


圖 3-7 文獻之 Hybrid DTM 系統架構圖[7]

這篇論文提出了一種應用在作業系統工作排程時的判斷機制，一般的軟體層在分配核心工作時，只會以每顆核心負載的工作資訊來判斷接下來是否要繼續派發工作，抑或是工作搬移，並未考量到熱平衡的管理，因此當高溫核心的工作負載低時，作業系統仍會判斷此核心處於閒置狀態而給予更多工作，導致高溫核心無法降溫，文獻除了將工作所造成的熱效應也一併考量在軟體層，也針對工作搬移的部分進行預處理，避免不必要的工作搬移而浪費效能。

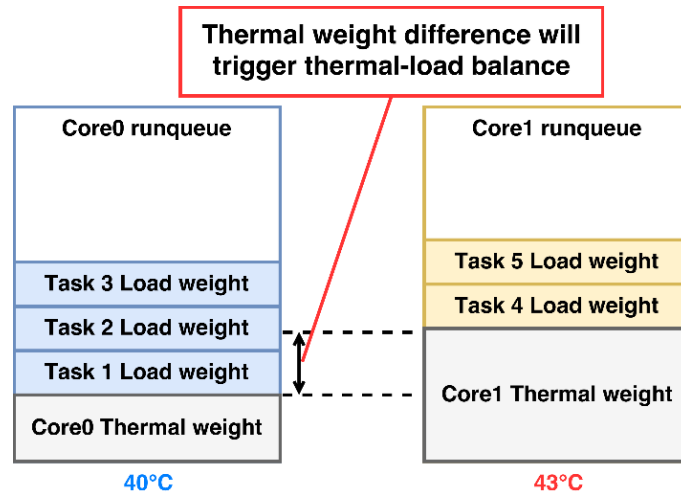


圖 3-8 文獻之決策熱平衡[7]

在硬體端的動態熱管理，此篇論文使用了透過作業系統來控制的溫度感知時脈閘控(Temperature-Aware Clock Gating)以及 DVFS 技術，由於封裝技術與佈局圖的設計，在不同位置的核心會有相異的升溫/冷卻速率，因此相同的工作分配到不同的核心所造成的溫差亦不相同，單純利用軟體端的工作搬移技術有可能讓某些核心完全呈現停擺的狀態，因為工作搬移技術而讓工作完全移到低溫核心，如此會造成過大的效能損失，因此這篇論文加入了溫度感知時脈閘控的技術，讓高溫核心在需要降溫時，除了靠工作搬移技術將工作派發給其他低溫核心外，也可以利用時脈閘控技術將溫度降低，同時又不減少太多效能。

而針對溫度感知的 DVFS 技術也在此篇論文提及，一般應用在多核心系統平台的 DVFS 可分為兩種：一種是全部核心同時使用同一種頻率電壓組合的 global DVFS，另一種則是針對每顆核心可以個別調整頻率電壓的 local DVFS，local DVFS 雖然相較 global DVFS 可以較靈活調整每顆核心的電壓頻率，但也相對付出較多的效能，此篇論文考量到 3D IC 核心的佈局擺放，提出了一種溫度感知的動態電壓頻率調整方法(Temperature Correlation-Aware DVFS(TCDVFS))，由於 3D IC 的核心位置會有水平與垂直的擺放方式，而上下垂直的核心其溫度相互影響的相關性相較於水平並排的核心來說較強，因此 TCDVFS 將上下層核心視為一組核心堆疊(core stack)，且以一組核心

堆疊為單位調整頻率電壓，如此能避免熱傳導相關性強的兩顆核心彼此的頻率電壓差距過大。

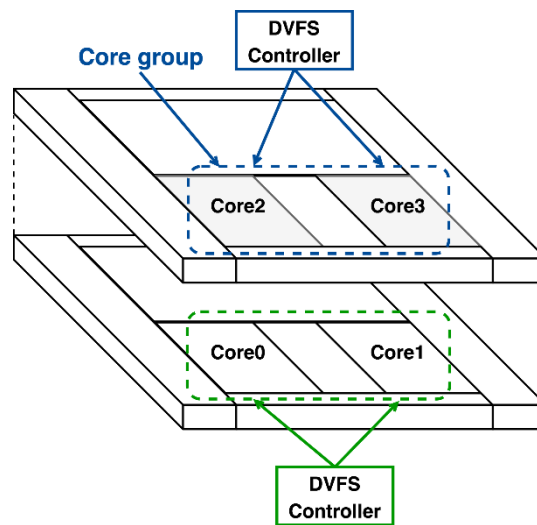


圖 3-9 文獻之 DVFS[7]



3.3 相關文獻總結

	LPHPLab[4]	ICMSAO,2019[5]	DATE,2015[6]	LPHPLab[7]
Target	GPGPU	MCSoc	CPU and GPU	CPU
Evaluation	GPGPU	CPU	Chip	CPU
Management Method	DVFS Clock Gating	Task migration DVFS	DVFS Turn on/off active cores	Task migration Temperature-aware DVFS
Main idea	Considering runtime hardware situation for power management	Uses both task migration and DVFS DTM techniques to improve the performance	Calculates a precise power budget to control the type of cores and frequency	Uses Thermal-load weight to predict Task migration thermal impact & Avoid over-balanced condition

表 3-1 相關文獻總結

這個章節之中，整理了四篇有關動態熱與功率管理的相關文獻，如表 3-1 所示，第一篇介紹了圖形處理器的功率節能方法，後三篇則探討多核心系統與異質架構平台上的動態熱管理方法，其目標都是為了在熱管理與效能上取得平衡，而使用了硬體與軟體端的低功率與熱管理方法，在這個小節之中將綜合以上幾篇論文的觀點，並且以下列幾點作為總結。

在四篇文獻中都有使用動態電壓與頻率調整(DVFS)作為低功率管理與降低熱效應的方法，可以觀察到不管是 CPU 多核系統或是 CPU-GPU 異構計算平台，針對 CPU 核心的溫度管理方式有很多種，從軟體層級來探討可以觀察到文獻[5]與文獻[7]都是使用了工作搬移技術，但相較後者有提前考量搬移後的結果，前者則是直接將工作搬移給未運作的核心，並且提出所謂的 dark silicon era 的模式，比起全部核心同時運作，利用較少的核心雖然會犧牲部分效能，但是在搬移工作的過程中比較不需擔心會發生過度搬移的結果，因為搬移工作的接收方是屬於 dark core，也就是因未使用而關閉的冷核心，而文獻[4]與文獻[6]則是針對硬體的使用率來選擇管理的手法，前者是利用

硬體特性與使用率來區分關鍵功率路徑與非關鍵功率路徑的元件，並藉由 DVFS 來調整，後者也是觀察核心的使用率，但較為不同的是以溫度為考量，計算該時間內的使用率與功率會不會對系統造成溫度超標，而這些動態熱管理較多是針對 CPU 核心，由於發配 CPU 工作的機制來自於作業系統，因此如果要控制 CPU 的工作排程可以選擇由軟體層級切入，如文獻[7]就是以及時溫度回饋來影響作業系統的工作排程，並且考量到不同方向間的核心彼此溫度影響的相關性。另外由於 CPU 可以使用工作搬移技術，讓不同核心間的工作可以自由調度而不會有錯誤，因此在熱管理上較為靈活，而反觀 GPU 的控管機制如文獻[6]，在異構系統平台上的 GPU 較常使用的手法為調整全體 GPU 元件的頻率，並未針對 GPU 上每顆串流處理器的高溫區塊或是低溫區塊來分別討論，且在工作搬移的部分，由於 GPU 本身的工作份量是以 workgroup 為單位，因此當串流處理器接收到 workgroup 後，其所處理的資料會在工作完成前暫存在自己的內部記憶體裡，且每個 warp 的及時狀態如 PC 等等與 warp 之間的資料關係性都暫存在元件內部，這導致如果要在 GPU 的串流處理器間實現工作搬移是一件不容易的事，而由於串流處理器間的 workgroup 可能來自於同一份 Kernel code，而一份 Kernel code 下的 Packet 需要等到所有 workgroup 都完成後才能進行下一份新 Packet 的 workgroup 派發，這導致停止正在工作的串流處理器可能會無法發送下一份 Packet 的 workgroup，連帶影響其他串流處理器的工作進度。

總結來說，通用計算圖形處理器上的熱管理不太適合進行工作搬移，如果想要停止串流處理器但又不希望影響到整體的工作流程，應該是在串流處理器完成自身所負擔的 workgroup 時為最佳時間點，因為停止工作的串流處理器沒有剩餘的工作資料殘留，因此關閉串流處理器不會影響到整體的工作流程，而停止串流處理器運作勢必會影響效能，這時如果能針對較低溫度的串流處理器適度增加頻率與電壓，剛好可以彌補效能而不至於超過最熱的核心溫度，因此整合以上這兩個關鍵重點，或許能建構出一套屬於通用計算圖形處理器的動態熱管理方法。

第 4 章 應用於通用計算圖形處理器的動態熱管理方法

在這個章節裡，首先會在 4.1 節中描述本論文所要解決的熱議題；在 4-2 節中則會介紹本論文所使用的模擬平台之架構與環境設定，其中包含了通用計算處理器模擬平台上的模組介紹以及建構功率與熱模型的相關方法；在最後的 4-3 節中會詳細介紹本論文所提出的動態熱管理方法，其中包含平衡熱溫差的手法與所搭配之工作排程的相關細節等

4.1 問題描述

如同在第二章中所提到的，由於近年來物聯網的應用與市場快速地成長，導致邊緣運算裝置的設計逐年受到重視，而應用在邊緣運算裝置上的圖形處理器則負責運行許多數據密集型的計算，由於 GPGPU 是由多核心及高平行化的架構組成，複雜的架構伴隨著龐大的功率消耗會導致嚴重的熱效應。雖有許多研究著重於 GPGPU 的效能提升與功率管理，但其中很少考慮到功耗對晶片的負面溫度效應，當晶片持續在過高的溫度上運作時會降低晶片可靠性與使用壽命，在效能上也會有所遞減，這些熱議題的資訊並不容易在晶片開發早期得知，因此很難對未知的熱分布做有效的熱管理，因此本論文提出了適用於通用計算圖形處理器的動態熱管理方法，其目的為平衡在系統運作期間逐漸增長的熱溫差並以降低晶片的最高溫度為考量，同時也安排了適當的工作排程機制來搭配動態熱管理的方法。

4.2 目標平台之環境與架構

本論文的目標平台為由成功大學計算機架構與系統實驗室(CASLAB)所開發之 CASLAB-GPUSIM[16][17]，在這個章節會介紹該平台的內部硬體架構與我們建置的相關功率與熱模擬模型。

4.2.1 平台介紹

CASLAB-GPUSIM 是一個針對 HSA 組織所訂定的規範而開發出的 GPGPU 運算模擬平台，其中所採用的指令集符合 HSAIL(Heterogeneous System Architecture Intermediate Language)規範的指令集，所謂的 Heterogeneous System Architecture 是由 HSA Foundation 所提出的異質運算開放標準[8]，其中 HSAIL 是一種中介語言，不同語言的程式原始碼會被先轉譯成 HSAIL，接著再轉譯成異質運算系統上各個運算單元的機器碼，如圖所示：

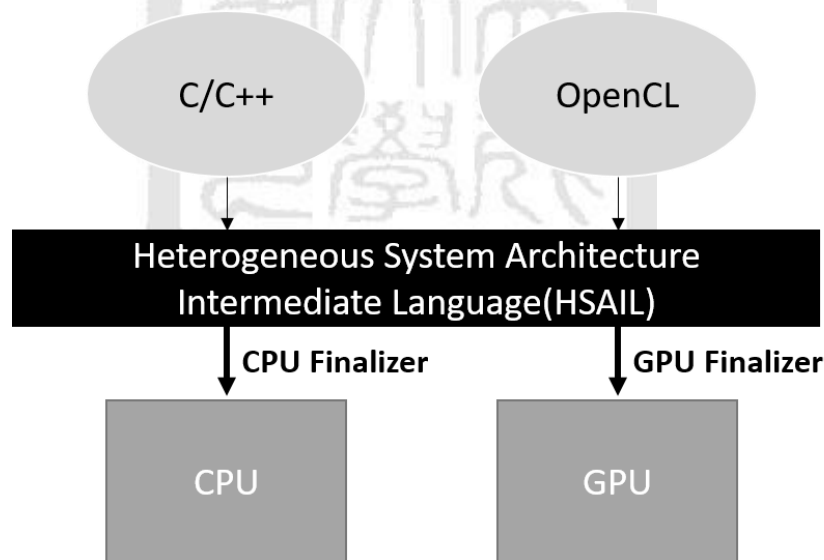


圖 4-1 HSAIL[8]

而 CASLAB-GPUSIM 整體平台的系統架構圖如圖 4-1 所示。在平台的軟體層是由自行開發之 OpenCL Runtime 以及 HSA Runtime 函式庫組成，包含了 CL Offline Compiler 以及 Finalizer。其中 CL Offline Compiler 是用來將 OpenCL 的 Kernel 程式轉換成 HSAIL 的形式，再透過 Finalizer 將 HSAIL 轉換為平台的自訂指令集，讓平台可

以執行 HSAIL 指令集。而在下方硬體層的部分是使用 System C 語言實作通用計算圖形處理器硬體，比起傳統設計流程，透過高階設計語言來建立虛擬平台能夠加速功能驗證的流程，而在 GPGPU 的硬體架構部份與 2.1 節中所描述的行為類似，當 Task Dispatch Unit 接收到軟體層的封包時，會按照特定的 workgroup schedule 分配 workgroup 給下層的串流處理器(SM)，接著每顆 SM 再依照 workgroup 內的相關資訊透過 Crossbar 和記憶體層的 DRAM 進行資料交換。

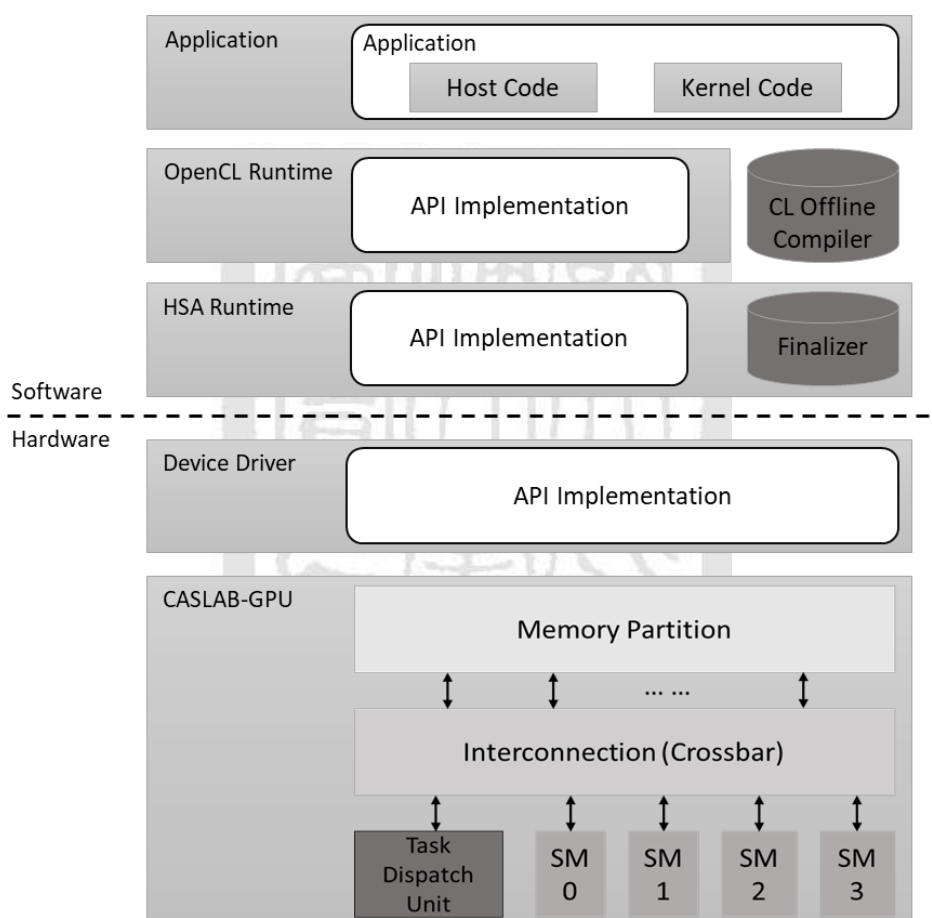


圖 4-2 CASLAB-GPUSIM 平台架構圖 [8]

4.2.2 多重串流處理器內部架構

多重串流處理器(SM)的內部架構如圖 4-3 所示，大致可以分成 Workgroup Initializer、Front-end Pipeline 和 Back-end Pipeline 三個部分，首先 Workgroup Initializer 負責接收 Workgroup 以及初始化的部分。Front-end Pipeline 負責指令的前處理與 decode，同時也掌管著 warp 的排程，最後 Back-end Pipeline 則是負責主要的運算以及 Load/Store 資料之相，接下來會針對內部的各個元件做介紹。

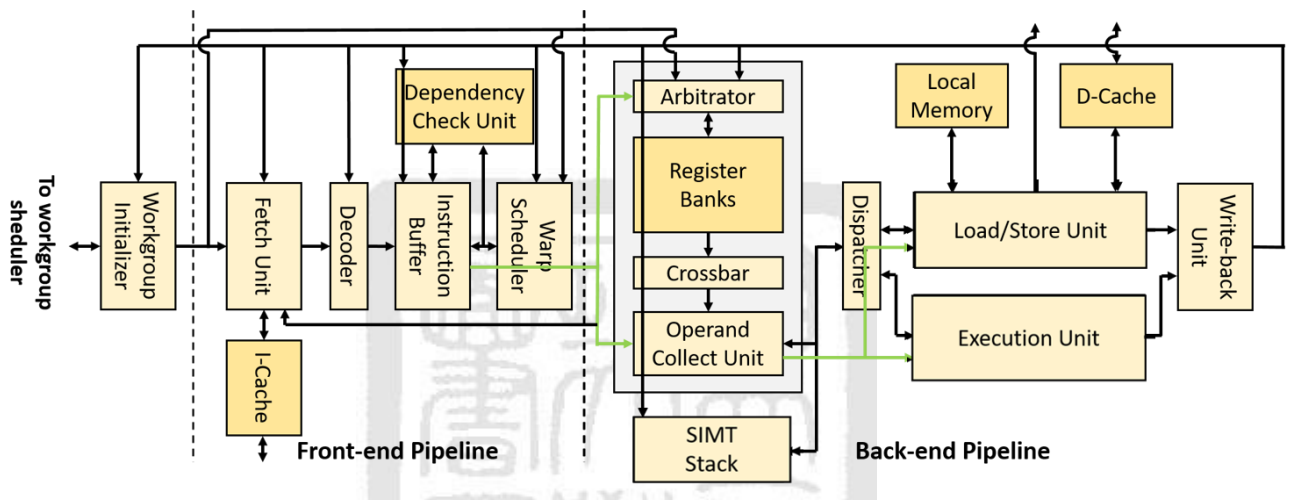


圖 4-3 多重串流處理器架構

Workgroup Initializer 作為接收 workgroup 的單元，當 Workgroup Scheduler 派發 Workgroup 到 SM 時會在這裡進行初始化的動作，並且負責管理整個 Workgroup 的進度與狀態，當 Workgroup 中的所有 Warp 都完成運算後便會釋放占用的資源並送出完成訊號。

Fetch Unit 為 SM 內部負責抓取指令的元件，在 Fetch Unit 中一共有 48 個 Warp 的 Program Counter (PC)，在 Workgroup Initializer 初始化之後會初始化各個 Warp 的資料，包含 PC、special registers、stack mask 等等，接著再將選定的 PC 傳到 I-Cache 中並抓取指令，傳到下一級的 **Decoder** 做指令解碼。

Instruction Buffer 為指令緩存單元，負責接收來自 **Decoder Unit** 的解碼指令，當指令傳入後便會被存放至屬於該 Warp 的 Instruction queue 中，最多可以暫存 48 個

Warp 的指令與 PC，接著會檢查在 queue 之中的各個指令是否符合派發的資格，當派發成功後便將其移出 Instruction queue。

Dependency Check Unit 是用來記錄和檢查暫存器中資料相依性(data dependency)的元件，包含 Read-After-Write(RAW)、Write-After-Read(WAR)、Write-After-Write(WAW)三種資料危障(Data Hazard)情況，由於 GPGPU 是採用 in-order issue, out-of-order execution 的方法，因此 Warp 內的相關資料必須先在此檢查，才能夠將運算資料傳往後級。被記錄在 Dependency Check Unit 之中的暫存器會在資料被寫回時被釋放。

Warp Scheduler 是負責派發 Warp 的單元，Warp Scheduler 會記錄 Warp 的 stall 資訊，此為判斷可否派發的主要條件，另外 Warp Scheduler 也會根據特定排程的機制來選擇一個能被派發的 Warp，而不同的排程機制如 Loose Round-Robin、Greedy-Than-oldest、Two Level 等等，皆有其排程的特色，而一個指令能被派發的條件包含是否通過 Dependency Check Unit 檢查，以及 Warp 是否被暫停，當選擇好要派發的指令後，派發訊號會傳至 I-Buffer 與 SIMT Stack 中取出該指令的相關資料，再等待資料到齊並且透過組合電路傳入 Operand Collector Unit 後才算完成指令的派發。

Operand Collector 屬於指令的緩存單元，包含 Execution Unit 和 Load/Store Unit 各別的收集器，從 I-Buffer issue 出去的指令會存放在 Operand Collect Unit 中，等待從 Register bank 拿到指令給 EXE/LSU 做運算或是記憶體存取。

Register Arbitrator 是決定讀取與寫入請求至 Register Bank 的控制單元，Arbitrator 會接收來自 I-Buffer 的讀取需求，當計算完讀取的運算地址後，再將需求訊號送至 register bank 級來取得 operand data。

Register Bank 是暫存資料的單元，根據 Register Arbitrator 所送來的請求進行資料的讀寫，並由 Crossbar 送出資料。

SIMT Stack 是用來處理分支與合併的元件，存放著每個 Warp 的 SIMD 遮罩 (SIMD Mask)，包含一些相關的資料如 PC 等等，這些資料是用於判斷 thread 寫回是否為有效的根據。

而在在後端管線的部份，主要以 **Execution Unit** 以及 **Load Store Unit** 來處理傳到後級的指令，Execution Unit 是運算核心單元，根據指令的不同，可分為 non-pipeline 指令(e.g. ADD、BRANCH、SHL…)以及 pipeline 指令(e.g. MUL、DIV、MAD…) 兩類，在此 GPGPU 平台上除了支援整數外也支援浮點運算型以及部分的特殊功能指令。而 Load Store Unit 則是針對記憶體讀寫需求發出存取指令的元件。在算出結果或是從記憶體讀取資料之後，就會透過 **Write Back Unit** 級將資料寫回。



4.2.3 功率模型

在了解計算通用計算圖形處理器的架構後，便可以針對平台內的各元件建立一套功率模型(Power Model)，如此便能得到每個元件在運作的當下所產生的功耗，並將其轉換為熱模擬器的輸入資訊進而得到即時的溫度回饋，在建立功率模型的方面，本實驗室(LPHP-LAB)將平台元件以記憶體類型與非記憶體類型來區分，記憶體類型的元件如 Register Bank Memory、Local Memory、L1 Cache、L2 cache 以及 D cache，將使用 ARM Memory Compiler 在 TSMC 40nm 製程下所產生的暫存器與 SRAM 硬體資訊來表示，並在功率模型中考量各記憶體類型元件的寫入功率、讀取功率與待機功率的數值，如此便可讓應用程式執行於 GPGPU 的虛擬平台時，根據當下硬體的狀況輸出相對應的功率，在 DRAM 的部分則是參考 Micron Technology, Inc.的 DDR4-2400 規格書[24]將相對應之功率資訊與延遲時間整合於功率模型與平台之中。而針對非記憶體類型元件的相關功率則是藉由硬體實作來取得功率資訊。整體的功率取得流程如圖所示，以下說明功率模型的建立流程：

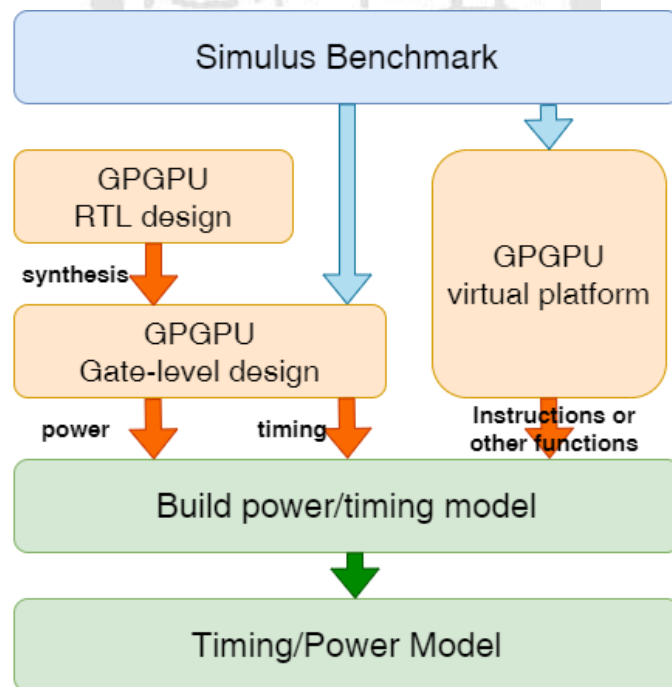


圖 4-4 非記憶體元件功率資訊取得流程圖

1. 首先在 System C 平台上運行應用程式(benchmark)，並針對個別在 System C 平台中的模組擷取出模擬時的波行檔(.vcd)，此波行檔包含模組在各時間的輸入輸出訊號，並利用 Python 語言將波行檔中每個時間點的訊號轉換成 Verilog 可以使用的測試訊號樣本，
2. 實現各塊模組於 RTL-level，且行為與時序上需與 System C 平台相同，再透過步驟 1 當中取得的測試訊號樣本來做為 RTL-level 的輸入訊號與輸出結果比較組，若發現有誤，表示 RTL-level 的模組未與 System C 平台上的模組功能相符，並持續修改 RTL-level 的模組到兩者行為相同為止。
3. 在步驟 2 驗證完 RTL-level 的模組與 System C 平台上的行為後，使用 Synopsys Design Compiler，在標準元件庫 TSMC 40nm 製程之下進行合成，可以得到 gate-level 模組，並重複使用步驟二當中的測試訊號樣本來驗證 gate-level 模組，同時使用 Synopsys Prime Time PX 進行功耗的分析。
4. 重複步驟 1 到步驟 3，在將所有 System C 平台中的模組轉換成 gate-level 模組並整合驗證後，將運行六隻應用程式所得到的平均功率資訊整合進功率模型之中。

藉由前述的功率模型建立流程，可以將 gate-level 的功率資訊紀錄在功率模型內，並在虛擬平台運作的同時輸出相對應的功率，下圖為運行在虛擬平台上的功率模型與每隻應用程式於 gate-level 模組上運作時的功率誤差，平均誤差約為 2.31%，這些功率可以用來表示目前平台的運作趨勢，並輸入到熱模型引擎中來得到即時的溫度資訊。

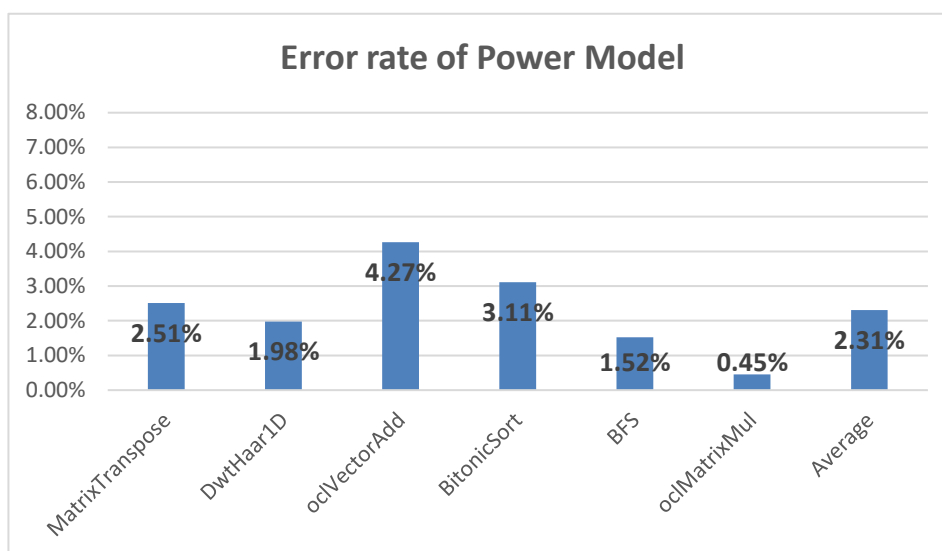


圖 4-5 非記憶體元件功率模型誤差

4.2.4 熱模擬引擎

在藉由功率模型得到 GPGPU 平台的即時功率下，這些資訊便可以回饋給熱模擬引擎來做使用，熱模擬引擎可以根據輸入的晶片佈局、所使用到的材質係數、熱傳導細數、以及佈局上的功率消耗，來模擬真實模型上的熱分布，熱模擬引擎在計算功率與溫度的關係時會因為使用不同種類的遞迴演算法而影響其效能，另外做熱分析時使用到的網格精度也會影響到熱模擬引擎所分析的精準度，統整上述說明後可以得知，熱模擬引擎需要考量兩個要點：熱模擬的速度以及切割的網格精度，在考量 GPGPU 平台其多核心的架構與複雜的平行處理機制下，本篇論文選擇文獻[25]作為熱模擬引擎的模型，相較其他的熱模擬器如 Hotspot，DG-Method 可以使用 CUDA Toolkit 來加速模擬效能，藉由平行計算來增快模擬的時間，且強大的熱模擬效能可以在相同的模擬時間下讓網格精度更加細緻，讓熱分析的結果更加準確。

在晶片佈局的部分，本論文實作了 16 核心多重串流處理器核心，在 Interconnection 的部分參考了 Advanced High Performance Bus(AHB)匯流排的相關資訊，並增加了控制工作排程機制的 Task Dispatcher Unit(TDU)及產生 DVFS 所需頻率的 PLL 與溫度功率控管元件(TPM)等等。其架構圖如下：

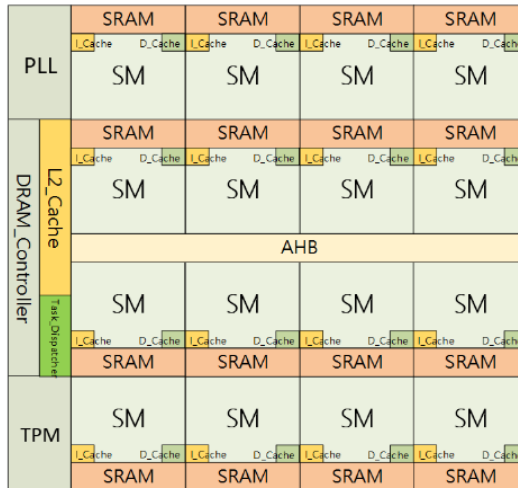


圖 4-6 使用於熱模擬引擎之 GPGPU 佈局

在冷卻機制的部分，除了本論文會使用到的動態熱管理機制來平衡熱效應外，在 16 核心的晶片上下層也同時佈有一層散熱金屬片，其目的是為了增加高溫核心的冷卻速率，如下圖：

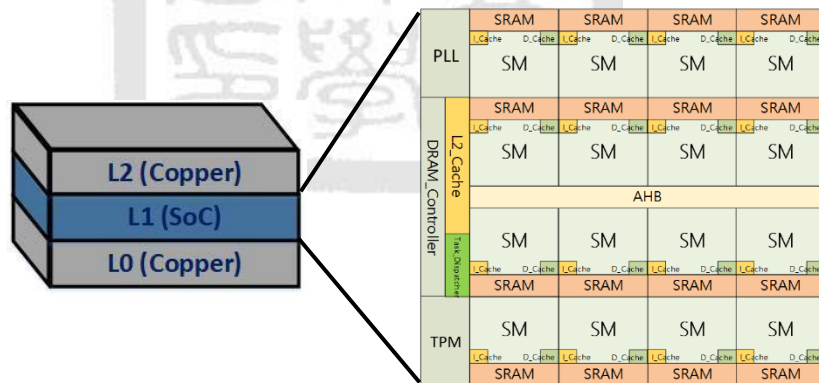


圖 4-7 使用於熱模擬引擎之 GPGPU 冷卻機制

4.3 動態熱管理機制

本篇論文會使用到前述 CASLAB-GPUSIM 的通用計算圖形處理器的平台，並搭配本實驗室開發的硬體功率模型與熱模擬引擎，提出動態熱管理機制以降低整體通用計算圖形處理器的熱溫差擴散與最高溫度。這個章節一開始會介紹動態熱管理如何平衡熱溫差的機制，第二部分則是介紹啟用熱溫差平衡方法的時間點，最後會解說搭配本篇動態熱管理的工作排程機制。

一般的動態熱管理可由所在的階層與行為區分成軟體端動態熱管理與硬體端動態熱管理，而時脈閘控與動態調整電壓頻率技術，這兩個技術在本篇論文中所使用到的硬體端動態熱管理技術。

4.3.1 熱溫差平衡機制

由於 GPGPU 平台上的核心會因佈局與不同的工作負擔而產生熱耦合，因此針對 GPGPU 平台上不同核心間的熱溫差分布，本論文提出了一種熱溫差平衡機制，可利用時脈閘控與動態調整電壓頻率技術，並搭配適當的 Workgroup Scheduler 來降低熱溫差，圖 4-8 是表示當平台上的核心間產生熱溫差時的狀況，圖中 Task Dispatcher Unit 內的 Packet Sender 會根據 Workgroup Scheduler 發配 workgroup 給每顆 SM 核心，而核心會因為其接收工作類型的不同，或是因為晶片佈局的緣故而提高溫度，此時就會產生熱溫差(Temperature gradient)。

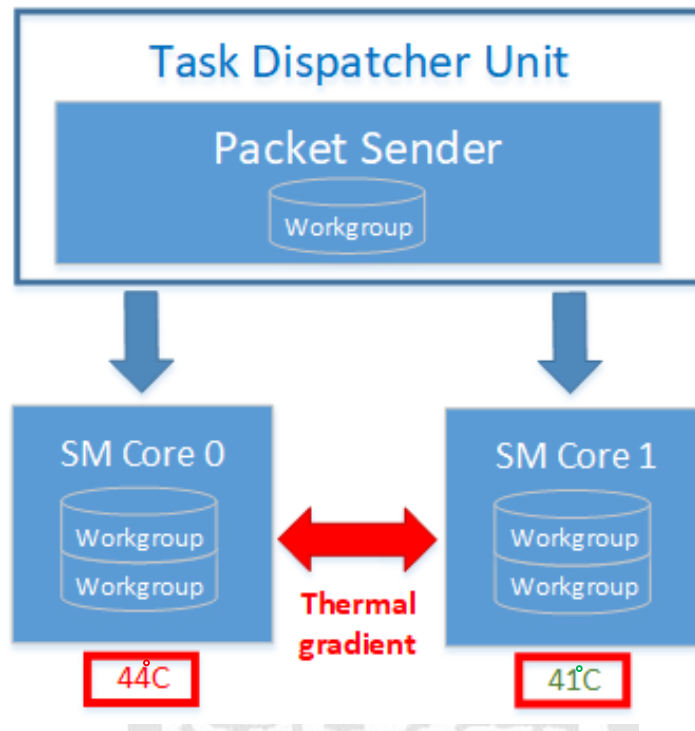


圖 4-8 熱溫差機制(一)

圖 4-9 是當熱溫差平衡機制被觸發時所應對的方式，首先在 Task Dispatcher Unit 的部分會讓 Packet Sender 暫時不派發工作給高溫核心，目的是準備讓該核心停止運作，而當高溫核心內部的工作結束後，動態熱管理元件會將高溫核心做時脈閘控來降低動態功耗，而針對低溫核心的部分，除了正常派發工作外，額外利用動態頻率電壓調整技術將低溫核心的頻率電壓調高，藉此平衡整體的核心溫度。

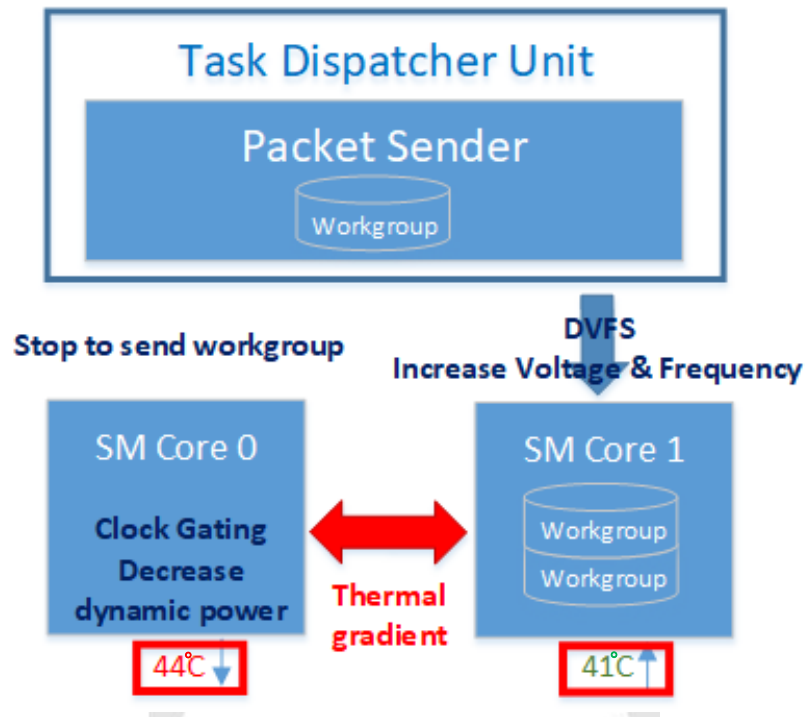


圖 4-9 熱溫差機制(二)

而圖 4-10 則是說明當熱分佈達到平衡時，Task Dispatcher Unit 內的 Packet Sender 會恢復派發核心工作的權利，而同時降低低溫核心的電壓頻率，使其正常運作。

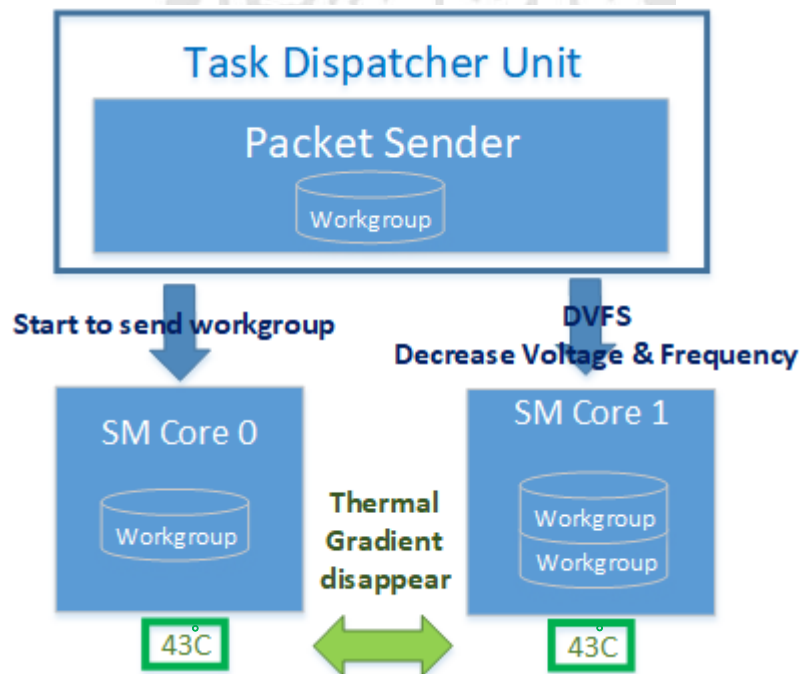


圖 4-10 熱溫差機制(三)

利用上述的熱溫差平衡機制可以在消除梯度的過程中讓高溫的核心降溫，由於停止發配 Workgroup 給核心意味著效能的降低，因此在平衡的過程中提升其他低溫核心的運作速度，如此不至於損失過多的效能，也加快了熱溫差的平衡時間。

4.3.2 考量工作負載之動態閾值機制

本論文依據上述設計於每顆核心之中的時脈閘控技術，以及動態調整頻率電壓的方式。停止高溫且已無工作負載的核心時脈，並調升其餘低溫核心的頻率電壓，藉此達到降低熱溫差的效果，可以知道啟動熱調節的關鍵在於出現過大的熱溫差，但多大的熱溫差門檻才需要啟動熱調節呢？由於漏電流所造成的靜態功率與溫度呈現正相關，因此溫度越高，靜態功率越大，因而造成溫度更快上升，因此當低溫的時間點出現熱溫差時，應該要適當使用動態熱管理技術來降低高溫核心的溫度，而在溫度即將超過安全運作的溫度前則需要強制降低運作核心的頻率與電壓，因此在這個小節我們會針對啟動熱調節的溫度節點做相關說明，並介紹相關頻率電壓的調整設定。

首先針對啟動熱調節的時間點，我們將使用平台上核心之間的最大熱溫差 ΔT 做為指標，所謂最大熱溫差是指在平台上最高溫度串流處理器與最低溫串流處理器的溫度差距，公式如下：

$$\Delta T = \text{The Highest core temperature} - \text{The Lowest temperature core} \quad (2.3)$$

當此差距突破預設閾值時便會觸發前小節所說明的熱溫差平衡機制，在此我們設計三個相關的閾值，分別為 $TBM_{throttle_start}$ 、 $TBM_{throttle_end}$ 與 $TBM_{throttle_max}$ ，下圖為閾值解釋與相關說明

ΔT	: the temperature difference between the hottest core and the coldest core
$TBM_{throttle_start}$	: the temperature difference threshold which will change state to Regulating State when ΔT is larger than $TBM_{throttle_start}$
$TBM_{throttle_end}$	: the temperature difference threshold which will change state to Balanced State when ΔT is smaller than $TBM_{throttle_end}$
$TBM_{throttle_max}$	: the maximal temperature difference threshold

圖 4-11 熱溫差閾值

首先， $TBM_{throttle_start}$ 是定義動態熱管理開始的熱溫差閾值，當 ΔT 超過這個閾值時便會開始動態熱管理，其次是 $TBM_{throttle_end}$ ，其為定義動態熱管理結束的熱溫差閾值，當 ΔT 低於這個閾值時便會停止動態熱管理，最後則是 $TBM_{throttle_max}$ ，表示動態熱管理中的熱溫差最高上限閾值。

利用以上的閾值，可以把整個熱溫差平衡分為三個狀態表示，分別為 Ordinary State、Regulating State 與 Balanced State。其流程如下圖所示：

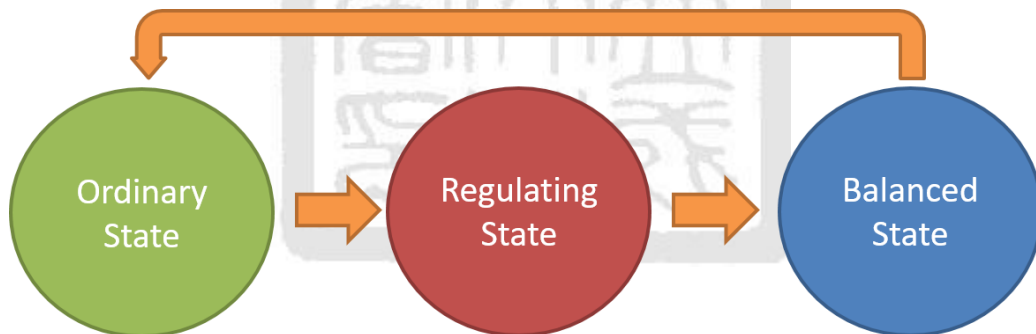


圖 4-12 平衡熱溫差流程

為了方便理解每個狀態所代表的涵義，以下使用了一組閾值的範例來說明：

ΔT	: the temperature difference between the hottest core and the coldest core.
$TBM_{throttle_start} = 1.0$	: the temperature difference threshold which will change state to Regulating State when ΔT is larger than $TBM_{throttle_start}$
$TBM_{throttle_end} = 0.9$	: the temperature difference threshold which will change state to Balanced State when ΔT is smaller than $TBM_{throttle_end}$

圖 4-13 熱溫差閾值範例

接下來的直線圖為 GPGPU 在運作過程中的即時狀態，橫軸為時間，以 10ms 為一單位，縱軸則是 ΔT 在運作過程中的數值，

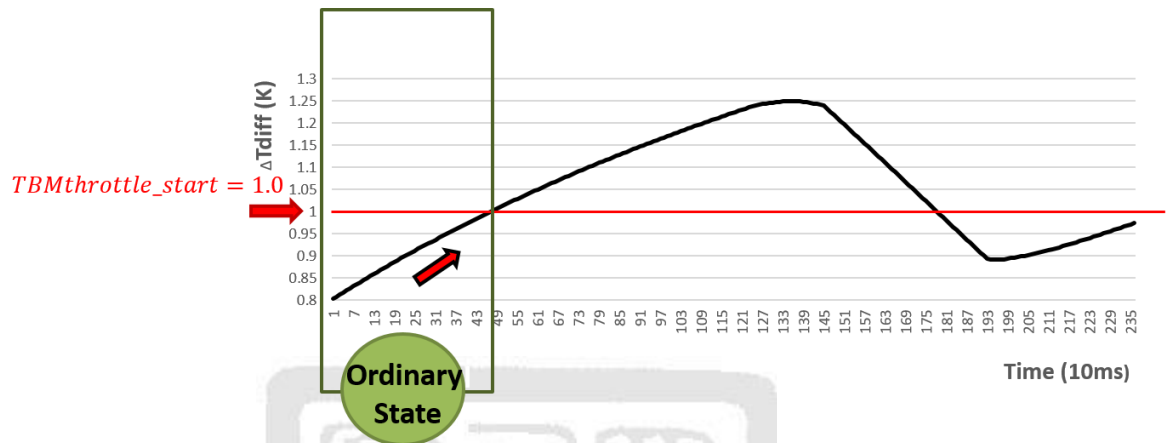


圖 4-14 熱平衡機制範例(一)

圖 4-13 是 SM 正常運作的狀態，在此我們稱為 Ordinary State，在一開始的 Ordinary State，由於最大熱溫差 ΔT 並未大於 $TBM_{throttle_start}$ 的閾值，因此 GPGPU 整體的工作流程不會被打斷，而由於每顆 SM 之間的指令差異與熱佈局的影響，可以觀察到 ΔT 的提高表示熱溫差正在持續上升。

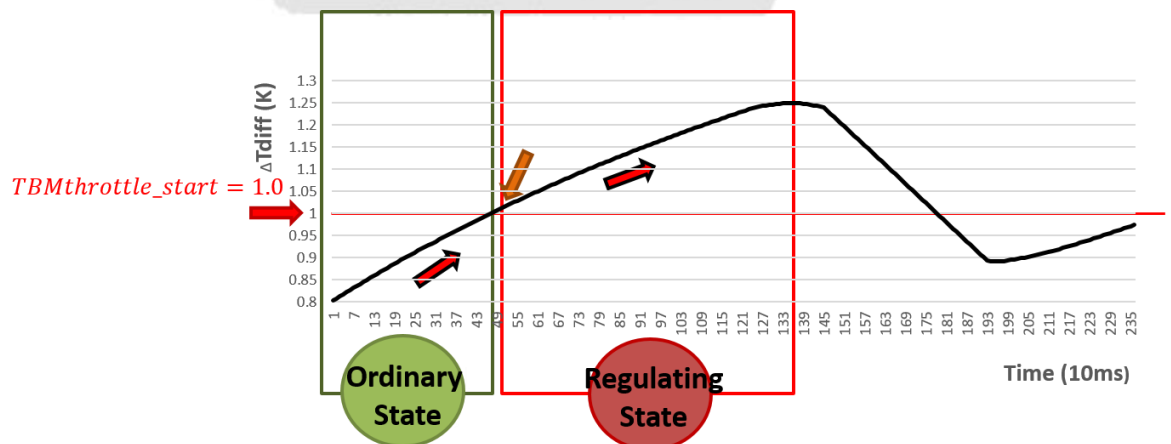


圖 4-15 熱溫差閾值範例(二)

圖 4-14 是當 ΔT 突破了 $TBM_{throttle_start}$ 的閾值時，表示此時可以開始平衡熱溫差，這個狀態我們稱之為 Regulating State，首先溫度管理元件 TPM 會通知負責發送 workgroup 給 SM 的 Task Dispatcher Unit，使其不再派發新的 workgroup 給高溫的 SM，而這時由於高溫的 SM 其內部的工作尚未做完，可以觀察到 ΔT 會持續的上升。

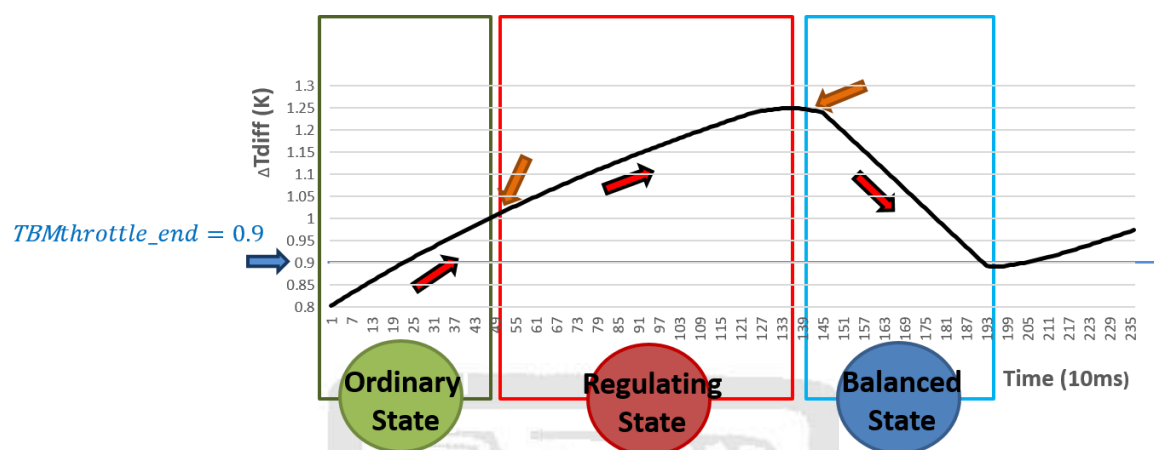


圖 4-16 熱溫差閾值範例(三)

圖 4-14 是當高溫的 SM 做完了本身內部的工作後，表示此時可以開始執行熱溫差的平衡，因此稱這個狀態為 Balanced State，在這個狀態內由於高溫的 SM 已經做完自身內部的工作，因此這時候 SM 本身的動態功耗會被 Clock gating 住，並且開始降溫，相反的對低溫核心來說，此時會利用 DVFS 技術將頻率與電壓調高，除了加速熱平衡之外，加速的過程不會產生最高溫度，亦可以用來彌補效能。

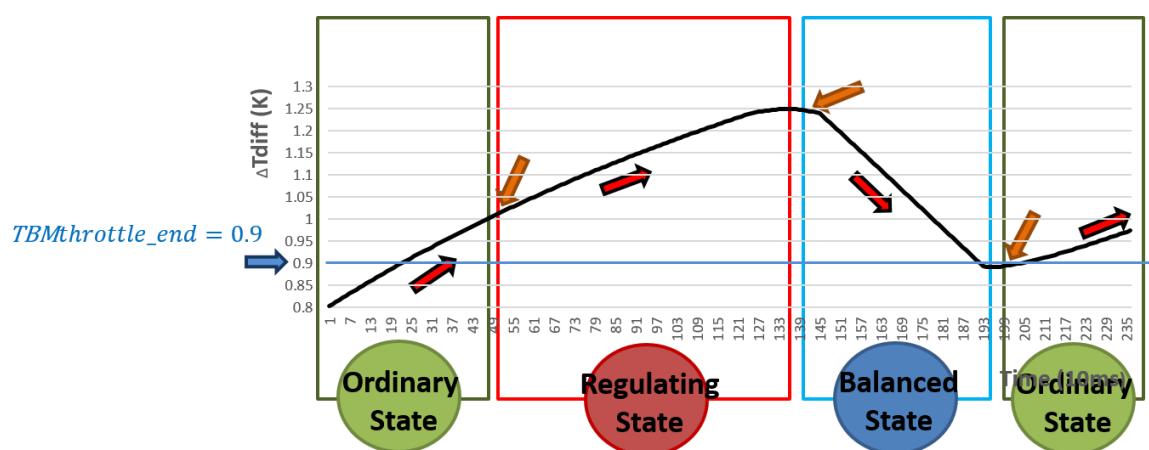


圖 4-17 熱溫差閾值範例(四)

圖 4-15 是當 ΔT 低於 $TBM_{throttle_end}$ 的閾值時，表示熱溫差以達到可接受的範圍，此時便提醒 Task Dispatcher Unit 恢復派發 SM 工作的權利，而同時降低低溫核心的電壓頻率，使其正常運作，並回到一開始的 Ordinary State。

而在設定熱溫差的閾值部份，若是採用一個定值表示熱溫差門檻，會發現效果最差，因為熱溫差最多只能降低到一個定值(由於散熱環境的限制)，如此一來會導致降溫的時間過長，造成過多的效能浪費，因此本論文提出了一種能夠針對當前溫度與功耗，動態調整適合的熱溫差門檻，能夠盡量在不損失過多效能的情況下使用熱節流管理。本論文預計所使用之溫度感知器其精度最小到 0.1 度，在 $TBM_{throttle_start}$ 與 $TBM_{throttle_end}$ 部份的原始設定為 0.5 與 0.4，而在 $TBM_{throttle_max}$ 的部份則是根據環境而設定為 1.0。如下圖所示

$TBM_{throttle_start} = 0.5$	: the temperature difference threshold which will change state to Regulating State when ΔT is larger than $TBM_{throttle_start}$
$TBM_{throttle_end} = 0.4$	: the temperature difference threshold which will change state to Balanced State when ΔT is smaller than $TBM_{throttle_end}$
$TBM_{throttle_max} = 1.0$	: the maximal temperature difference threshold

圖 4-18 熱溫差閾值

而在調整熱溫差的閾值部份，其遵守下圖的規則，當 ΔT 小於 $TBM_{throttle_max}$ 且 ΔT 在 Balanced State 時上升時，表示此時的 $TBM_{throttle_max}$ 對目前的模擬樣本來說過小，無法在 Balanced State 降低溫度，因此整體的熱節流閾值可以上升 0.1 的幅度來讓溫度平衡機制正常運行，而若在 Ordinary State 中， $\Delta T < TBM_{throttle_max} - 0.1$ 時，表示此時熱節流閾值相對於目前模擬來說過大，此時會嘗試將整體的熱節流閾值下降 0.1，藉此觸發溫度平衡機制來降低溫度。

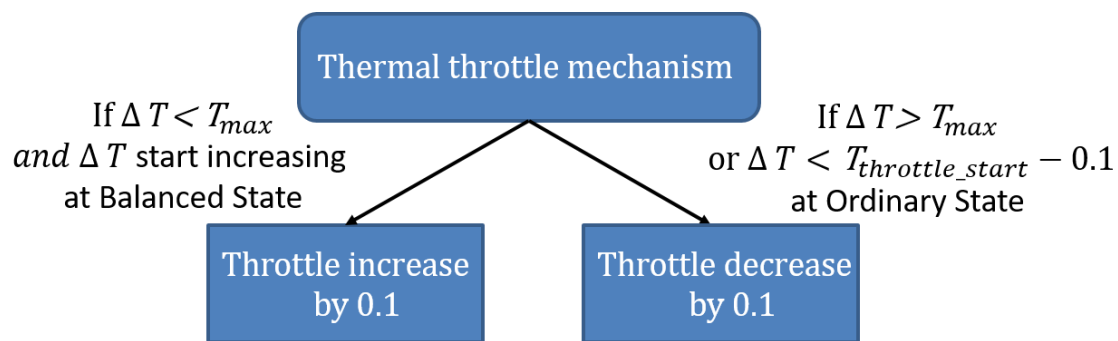


圖 4-19 動態熱溫差閾值

4.3.3 工作排程機制

由於關閉多重串流處理器會暫停核心上執行到一半的負載工作，進而影響 Packet Sender 繼續發配工作的流程，因此在熱節流的調整之前，我們需要提早讓多重串流處理器完成其核心上的工作，並停止讓上層的 Packet Sender 繼續派發工作下來，此時一旦多重串流處理器完成工作的時間過長，很有可能導致核心超過熱節流的溫度，因此需要盡量讓多重串流處理器縮短完成工作的時間，而這取決於 Packet Sender 分配給每個核心的工作負載，一般來說為了要提升 Memory latency，Packet Sender 會派給多重串流處理器能夠負荷 warps 的最大 workgroup 數量，但這時須考慮到多重串流處理器本身 pipelined stall 的機率，造成多重串流處理器處於 pipelined stall 有許多可能性，研究顯示當多重串流處理器同時執行過多的 workgroup 時，有機會提昇 pipelined stall 的問題，如圖 4-18 左所示，當上方的 SM 0 所運行的 workgroup 數量為 4 個，而 SM 1 所運行的 workgroup 數量為 3 個時，對記憶體區讀取的資料流會堵塞，造成多重串流處理器處於 pipelined stall，而圖 4-18 右則是選取過少的 workgroup 數量，這樣反而會導致多重串流處理器本身隱藏 memory latency 的能力下降，表示多重串流處理器可以存取更多的 workgroup 數量。因此選定一個小於多重串流處理器能最大負荷的 workgroup 數量，不僅能夠在需要停止多重串流處理器時縮短完成工作的時間，同時也能降低 pipelined stall 的情況，

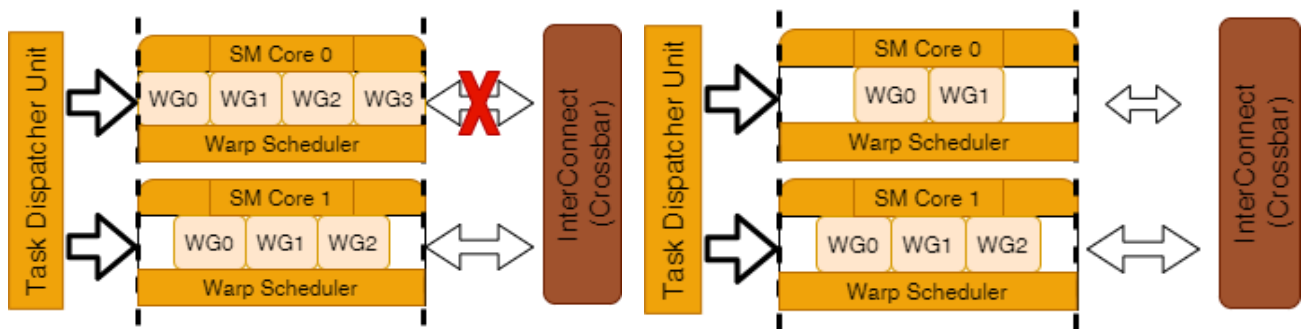


圖 4-20 對不同 wg 數量於 SM 內部影響

本篇論文加入了能夠偵測合適工作數量給多重串流處理器的工作排程機制，其偵測機制如下：

1. 在 Packet Sender 發送 workgroup 的一開始，先使用 LRR(Loose Round-Robin)的方式派發 workgroup，並給予每顆多重串流處理器能夠負荷 warps 的最大 workgroup 數量。
2. 此時每當多重串流處理器結束一個 workgroup 的運算，便紀錄這段時間多重串流處理器的 issue ratio。
3. 選取生成最大的 issue ratio 所對應的 workgroup 數量當作限制 Packet Sender 給多重串流處理器的最大 workgroup 數量。

由於多重串流處理器會根據其 Warp Schedule 內部的策略做指令的派發，為了確保每段 issue ratio 所派發的指令都是與多重串流處理器所包含的 workgroup 數量相關，因此本論文在 Warp Schedule 的部分採用 GTO(Greedy-Than-Oldest)來派發指令，在每個 Cycle 內，GTO 會先檢查前一個被派發的 Warp 是否仍為可以被派發的狀態，假如是的話就依然選擇同一個 Warp 繼續派發，直到 Warp 因為相依性等問題無法繼續被派發為止。一旦無法連續派發時，GTO 便會從當前準備完成的所有 Warp 中，選擇最早進 SM 的那個 Warp 來派發，在這個機制下會發現 SM 盡量以同一個 warp 做發配的考量，因此每個階段所呈現的 issue ratio 會與其內部所包含的 workgroup 數較相關。

使用此機制的主要目的並不是在加速整體運作的效率，而是為了讓熱平衡的時間點提前發生，所以選擇較少派發的任務數量，同時也考慮到 issue ratio 的影響，在兩者中做權衡。

4.3.4 動態熱管理之流程圖

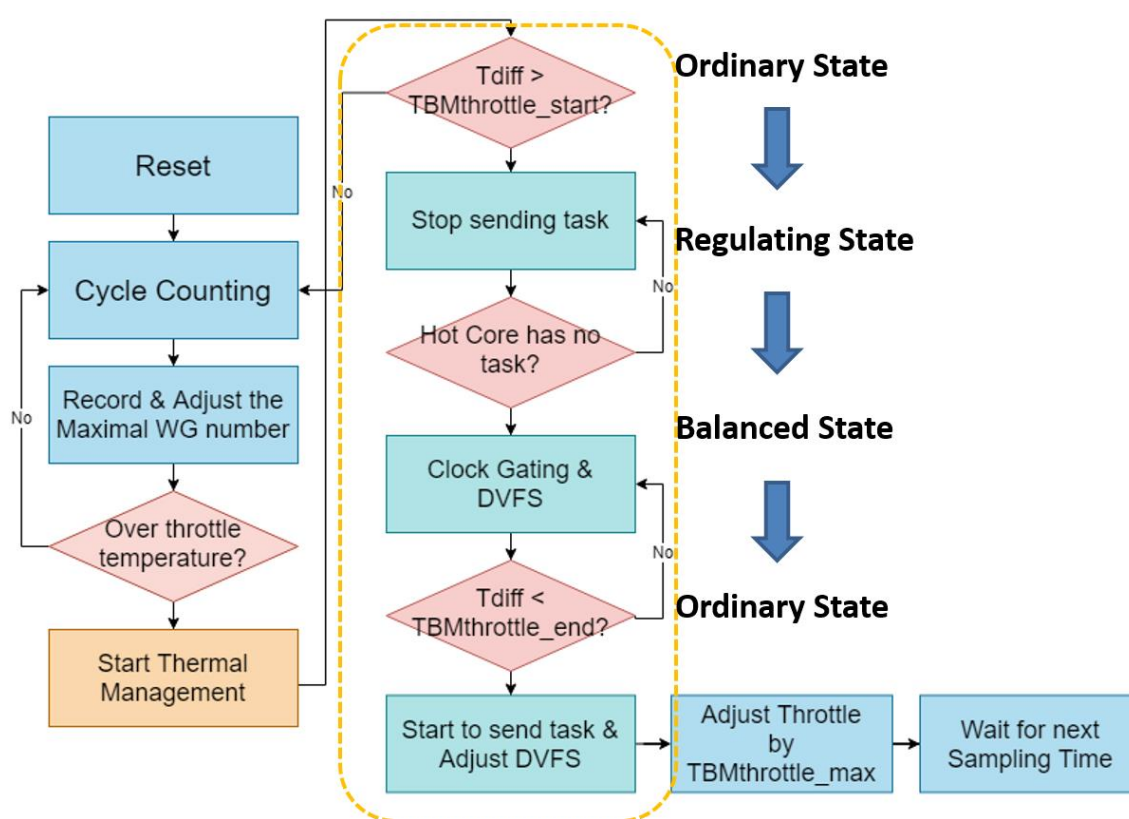


圖 4-21 動態熱管理演算法流程

整體動態熱管理演算法流程如上圖 4-21，當 Reset 結束後，便開始針對 SM 內部的 workgroup 數量做討論，此時如果保持在正常溫度，表示暫時不需要動態熱管理，而當溫度超過閾值時，流程便來到了中間的區域，也就是主要的熱管理方法，可以看到此處標有前述的熱管理方法的三個狀態，在分別經過 Ordinary State、Regulating State 與 Balanced State 後，表示熱平衡的流程結束，此時會根據剛剛管理當中的條件來調整相關的閾值，並等待下次熱管理的發生。

第 5 章 實驗結果與分析

5.1 實驗環境設置

本論文是以 CASLAB-GPUSIM 作為通用計算圖形處理器的模擬平台，硬體配置參數如下表

表 5-1 CASLAB-GPUSIM 之實驗環境

CASLAB-GPUSIM	Ver.PolyBench
ISA	HSAIL
SM core number	16
Max #Warps	48
SIMD Lane Width	32
RegFile Size	1MB
L1 I-Cache Size	32KB
L1 D-Cache Size	16KB
DRAM	Micron DDR4 2400

由於本論文會使用到動態電壓頻率調整的機制，因此在 CASLAB-GPUSIM 中需要加上 PLL(Phase-Locked Loops)來調整 SM Core 及 DRAM 端的頻率、DC-DC converter 調整電壓值、以及一塊 TPM (Thermal & Power Management Unit)來管理整體頻率電壓與控管 Task Dispatcher Unit 內的工作排程。與頻率電壓相關的實驗設定與架構圖如表 5-2 與圖 5-1 所示

表 5-2 CASLAB-GPUSIM 平台之頻率電壓設定

Thermal Management Related Setup	
SM clock rate (Default)	200 MHz
DRAM clock rate (Default)	200MHz
SM frequency level	250, 200, 100, 66.67 MHz
DRAM frequency level	250, 200, 100, 66.67 MHz
DC-DC converter	0.9-0.81 V

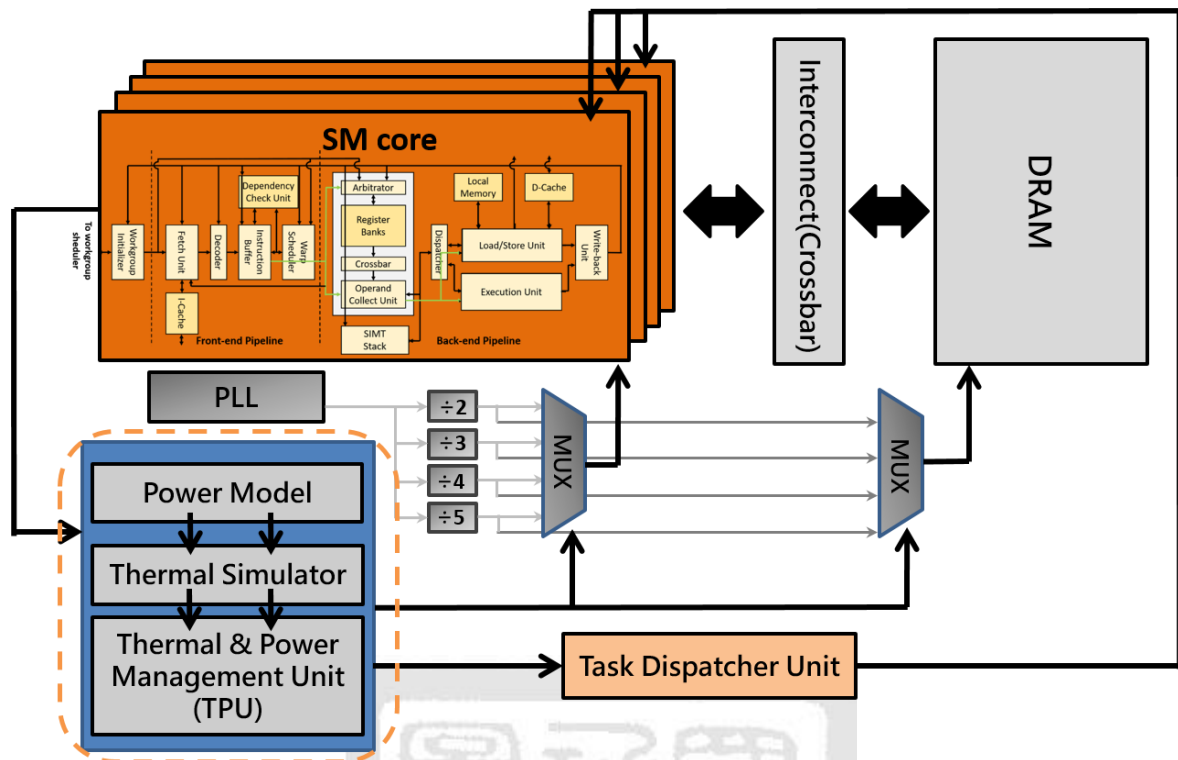


圖 5-1 功率與溫度管理架構

5.2 實驗結果

本章將使用了三組不同的演算法來比較各個性能指標，比較各組結果之 Peak Temperature、Performance、Performance Overhead、Energy、Average Power 等等

使用於實驗中的三組不同演算法介紹如下：

1. Baseline

第一組組別為 GPGPU 平台上的原始版本，並未加上任何動態熱管理與功率管理等演算法，此組是比較中的基準組別。

2. Hardware Runtime Utilization(HRU):文獻[4]

第二組組別是前面 3.1.1 節介紹的功率管理演算法，以各顆核心的硬體即時使用率 Hardware Runtime Utilization(HRU)作為基準，針對當下的運作模式調整核心與記憶體的电壓，以達到節能的效果。

3. Proposed Design:動態熱平衡技術:

第三組組別是使用本篇論文的演算法，強調的是在低溫的部份做動態熱平衡，而在溫度到達 80°C 時則開始做降頻與降壓的動作。

由於本論文的動態熱管理演算法的溫度閾值為 80°C，由於目前所使用之虛擬平台模擬器的真實模擬時間過久，考量到模擬時間與室溫的熱效應較不嚴重的情況下，兩組實驗的開始溫度決定由 60°C 開始模擬。而實驗所使用到的散熱佈局皆為 4-2-4 提到的散熱金屬片，安置在晶片的上下兩層，

5.2.1 不加入閾值限制之溫度管理比較

在實驗一中，會比較在沒有 80°C 的溫度閾值限制下，動態熱管理演算法的效能分析，此實驗一共有三組比較組，分別是原始設定之 Baseline 版本，Hardware Runtime Utilization(HRU)文獻版本，以及 Proposed Design，所使用的 Benchmark 依照其運算特性區分為三類，分別為 Computing intensive benchmarks、Memory intensive benchmarks 以及兩者混合的 Memory & Computing benchmarks 這三種。以下會分別使用這三種類型的 benchmarks 做模擬。

在 Computing intensive benchmarks 的部分，測試樣本屬於數據密集計算型，使用到三隻 benchmarks : 2MM、3MM、3DCONV 並串接模擬(2MM→3MM→3DCONV)。

圖 5-2 是三組演算法在模擬結束後的結果

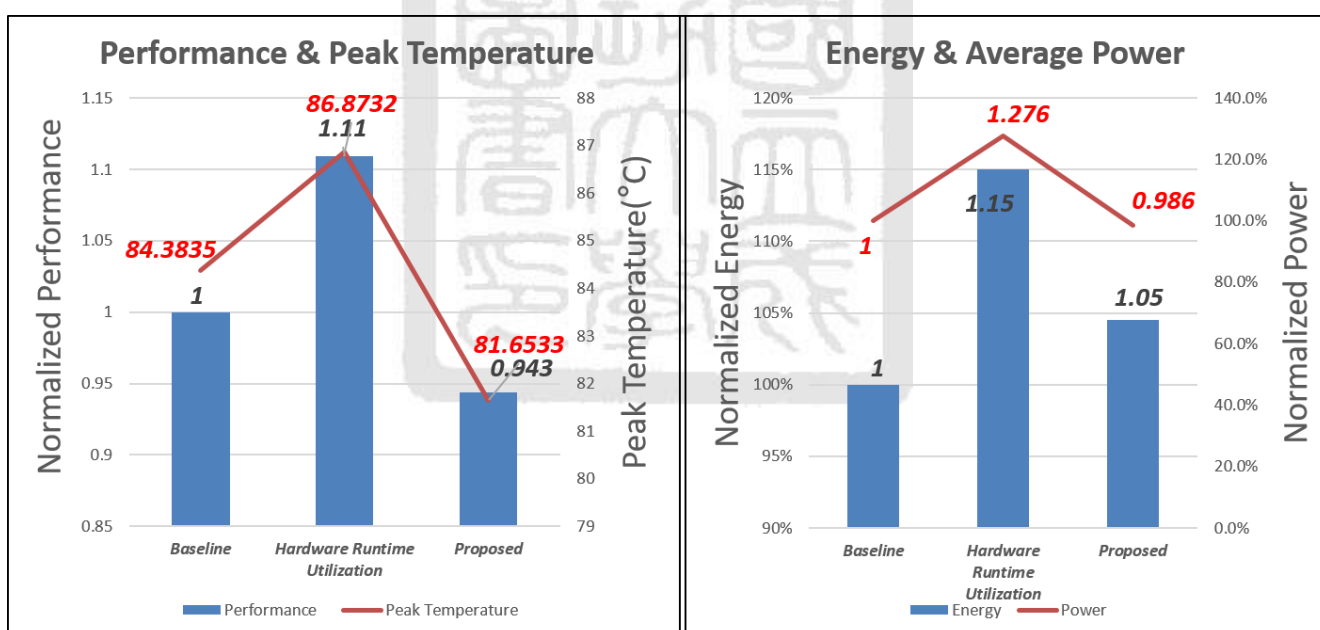


圖 5-2 針對數據密集計算型之測試樣本之實驗結果比較圖

由圖 5-2 可以看到在效能部分，由於 HRU 判斷了目前的 Benchmark 屬於數據密集計算型，因此在提升核心頻率與電壓後，效能相對其他兩者來說較好。但峰值溫度的部分相較 Proposed Design 來說也高了約 5 度左右，在節能方面也沒有發揮明顯的

效果。而雖然在效能與能量耗損上有些微損失，但相較其他兩者的溫度而言降低了約 3~5 度的效果，顯示了數據密集計算型的測試樣本上的控溫效果。

在 Memory intensive benchmarks 的部分，測試樣本多屬於記憶體存取型計算，使用到三隻 benchmarks : BICG、MVT、GEMM 並串接模擬 (BICG → MVT → GEMM)。

圖 5-3 是三組演算法在模擬結束後的結果，依序比較 Peak Temperature、Performance、Energy 與 Average Power。

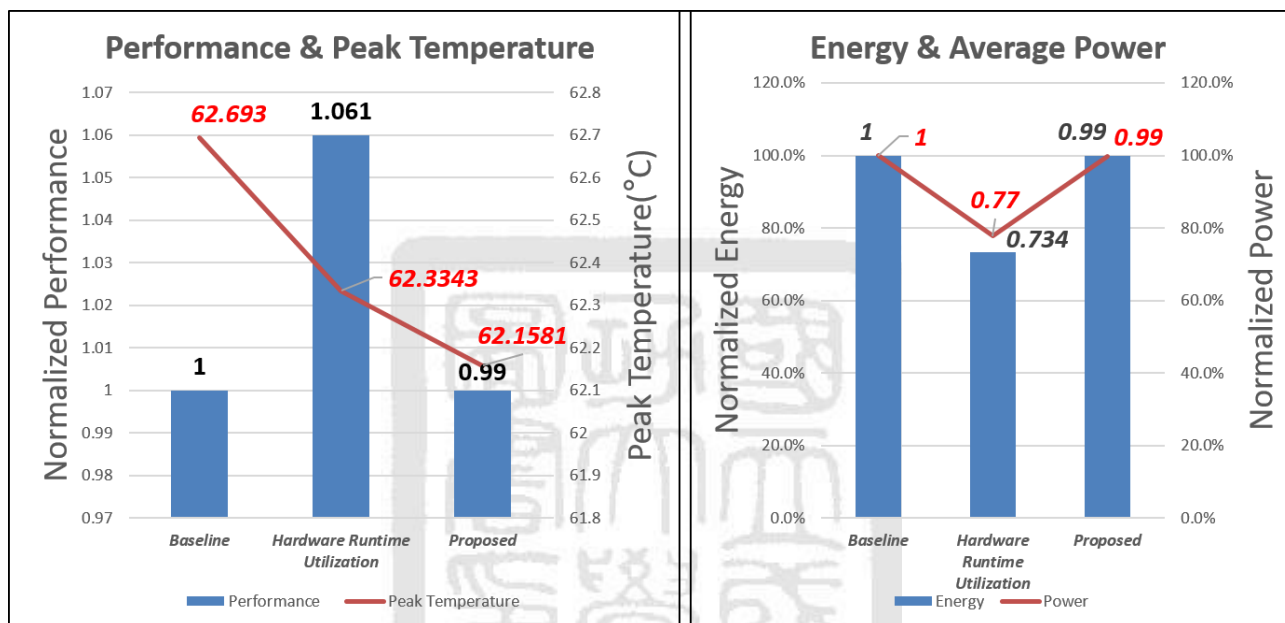


圖 5-3 針對記憶體存取型之測試樣本之實驗結果比較圖

由圖 5-3 可以看到在效能部分，HRU 相對兩者來說除了能提升約 5% 的效能外，在功耗部分 HRU 也能夠達到節省能量的效果。而 Proposed Design 能夠在效能與功耗上與 Baseline 的版本幾乎相同德強況下，讓峰值溫度比兩者都來的低，這是因為工作排程可以參照核心的溫度做工作派發，因此降低的溫度雖然不如數據密集計算型的模擬結果的多，但也可以達到部分的控溫效果。

在 Memory & Computing benchmarks 的部分，測試樣本混合了數據密集計算型與記憶體存取型計算，使用到三隻 benchmarks :GEMM、MVT、2MM 並串接模擬 (GEMM → MVT → 2MM)。圖 5-4 是三組演算法在模擬結束後的結果，依序比較 Peak Temperature、Performance、Energy 與 Average Power。

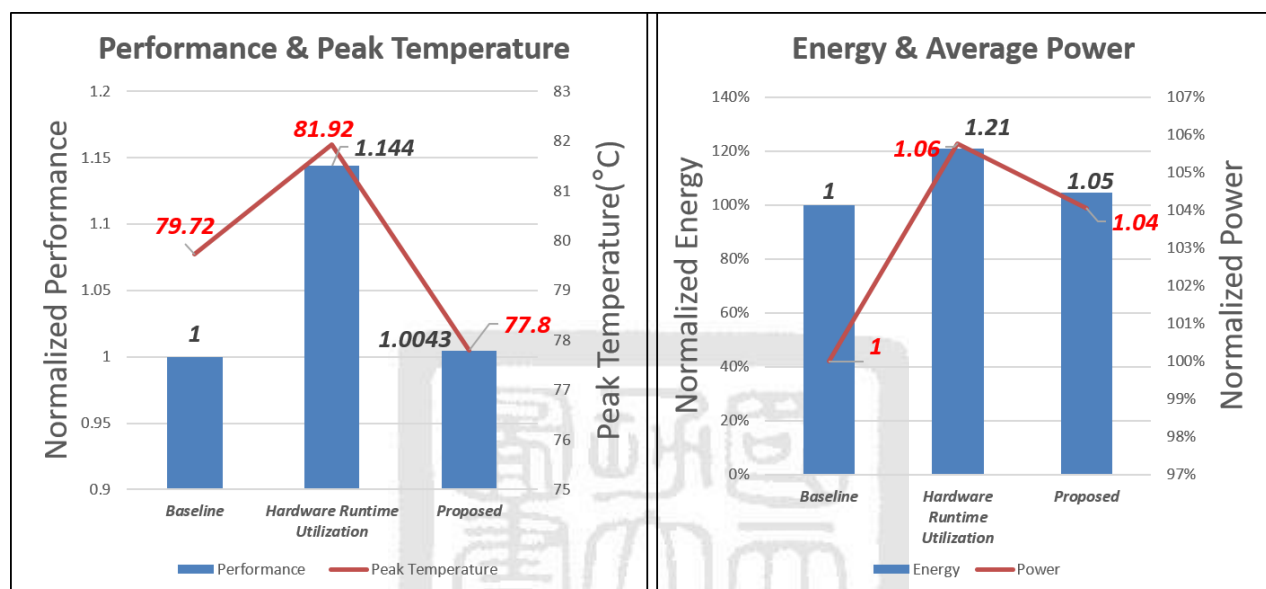


圖 5-4 針對混合型之測試樣本之實驗結果比較圖

由圖 5-4 可以觀察到在混合型的測試樣本中，結果與數據密集計算型的模擬結果雷同，在溫度方面 Proposed Design 能夠在損失 5% 的能量的情況下，將峰值溫度控制在比其他兩者低約 2~4 度左右。

5.2.2 加入閾值之溫度管理比較

在實驗二中，本實驗會以 80°C 作為溫度閾值的上標，當超過這個值後，DVFS 會強制將所有 SM 的頻率調低，此時 SM 會被強制降溫，而此以 78°C 為溫度閾值的下標，表示當 SM 降溫到這個溫度時，可以恢復工作，此時 DVFS 便反向處理，以這樣的演算法來限制溫度超標 80°C 。

在測試樣本的部分則依照應用趨勢的不同來分成兩組模擬，第一組測試樣本是先執行記憶體存取型再串接數據密集計算型的應用。第二組測試樣本則相反，是先執行數據密集計算型的應用再串接記憶體存取型的測試樣本。

在第一組的測試樣本中，按照上述的應用將六隻 Benchmark 做串接(MVT → 2DCONV → ATAX → 3DCONV → 2MM → 3MM)。表現出記憶體存取型再串接數據密集計算型的應用，圖 5-5 是三組演算法在模擬後依序比較 Peak Temperature、Performance、Energy 與 Average Power 的結果，圖 5-6 則是將模擬的時間與峰值溫度作圖的結果。

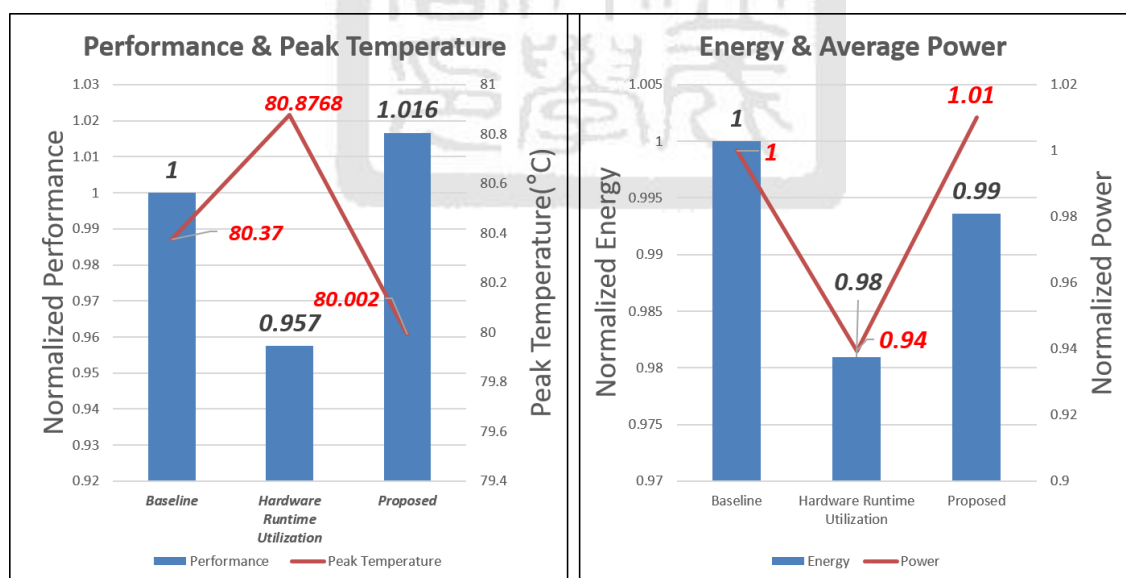


圖 5-5 記憶體存取型串接數據密集計算型之實驗結果比較圖

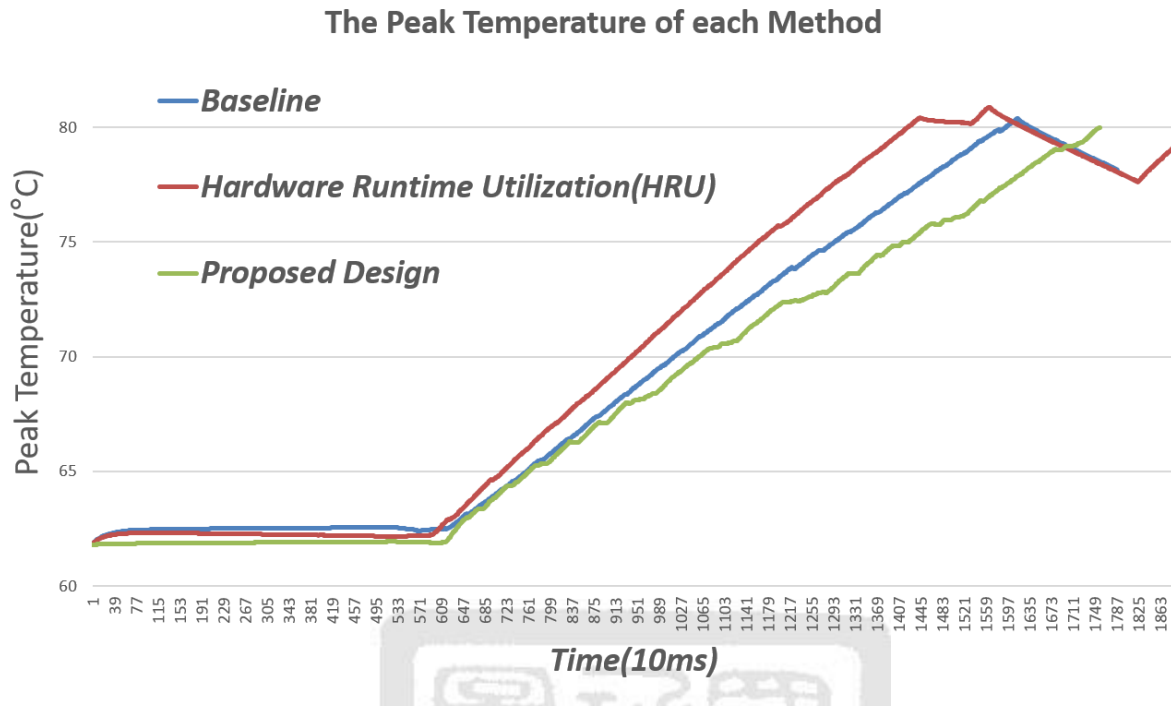


圖 5-6 記憶體存取型串接數據密集計算型測試樣本之直線圖

由圖 5-5 可以觀察到在記憶體存取型串接數據密集計算型的測試樣本中，HRU 的效能為三者中最低，主要是因為在超過 80 度的溫度閾值限制下，效能會被降低的頻率與電壓所牽制，而 Proposed Design 的部份除了在效能與能量耗損比 Baseline 版本好外，峰值溫度也是三者中最低，如果搭配圖 5-6 可以看出溫度上升的趨勢以 HRU 最為劇烈，其次是 Baseline，最後則為 Proposed Design。且可以看出本論文中的 TBM 設計能趨緩溫度上升幅度的現象。

在第二組的測試樣本中，按照上述的應用將六隻 Benchmark 做串接(MVT →2D CONV →ATAX →3D CONV→2MM→3MM)。表現出記憶體存取型再串接數據密集計算型的應用，圖 5-7 是三組演算法在模擬後依序比較 Peak Temperature、Performance、Energy 與 Average Power 的結果，圖 5-8 則是將模擬的時間與峰值溫度作圖的結果

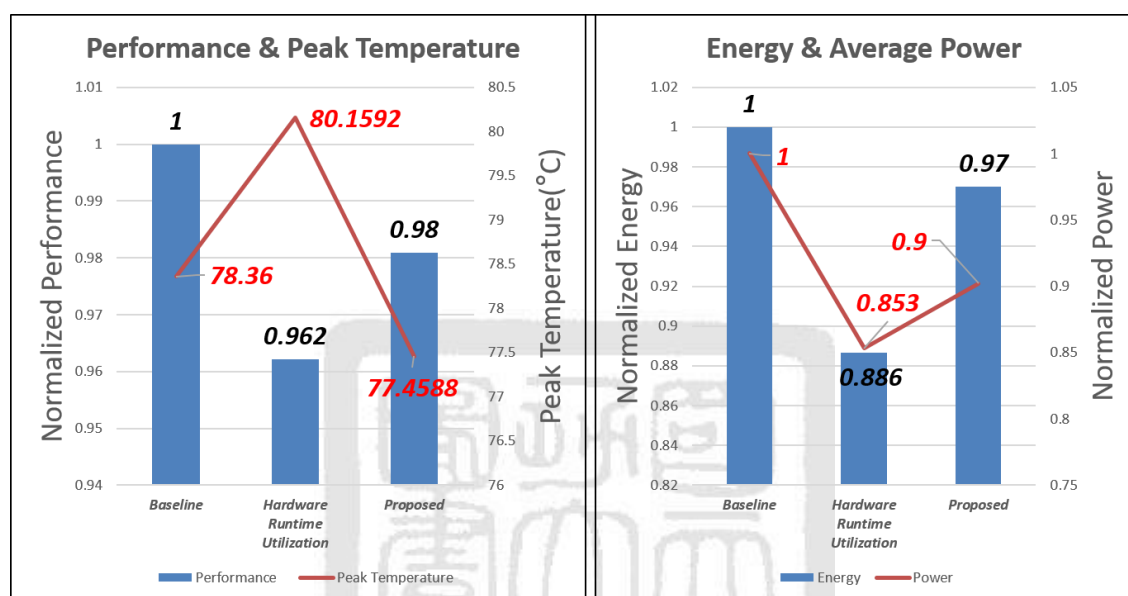


圖 5-8 數據密集計算型串接記憶體存取型之實驗結果比較圖

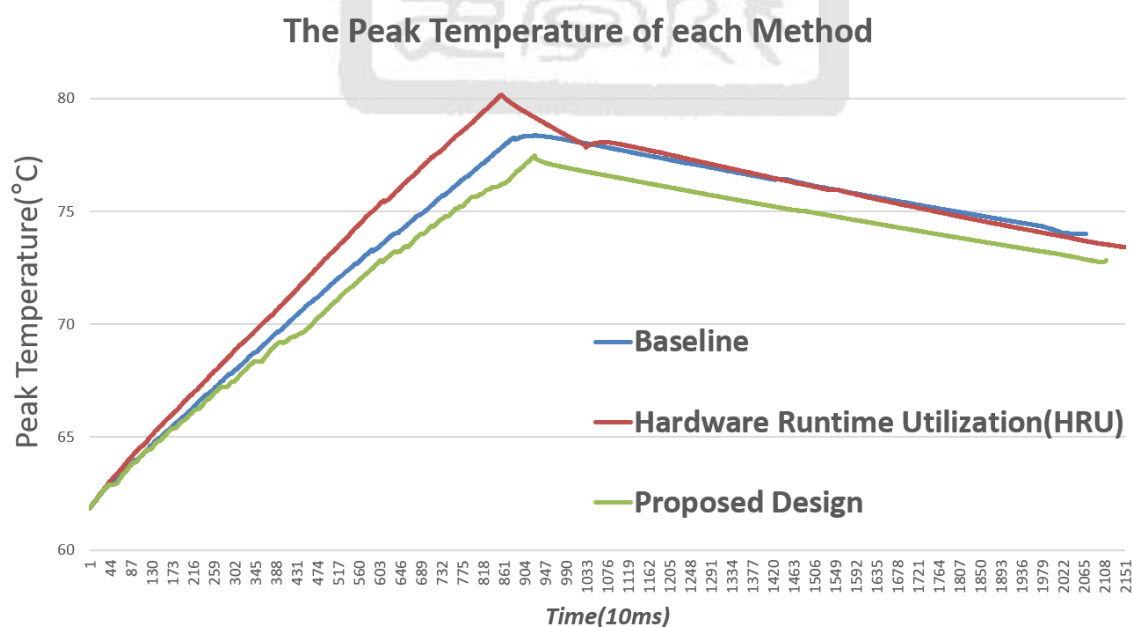


圖 5-7 數據密集計算型串接記憶體存取型之實驗結果比較圖

由圖 5-7 可以觀察到在數據密集計算型串接記憶體存取型的測試樣本中，Proposed Design 的峰值溫度可以控制在比其他兩者低 1~3 度左右，且相對 Baseline 而言能節省能耗約 3% 左右。而 HRU 雖然有發揮節能效果，但溫度與效能都是兩者內相對較差的。在圖 5-8 的部份，可以觀察到在執行數據密集計算型的測試樣本時，三者演算法的上升趨勢與前者實驗相同，在模擬後半段執行記憶體存取型時可以看到由於應用偏向記憶體存取取向，因此三者的溫度都有下降，由於 Proposed Design 相對於其他兩者來說溫度一直處於較低的狀態，因此在高溫區段中所受到的漏電流影響也較少，因此相較 Baseline 的版本而言較省能量。

Experiment I (No thermal constrain) :

Experiment II (Has thermal constrain) :

Computing intensive Benchmark				Memory intensive to Computing intensive benchmark			
	Baseline	HRU	Proposed		Baseline	HRU	Proposed
Peak Temperature	84.38	86.87	81.65	Peak Temperature	80.37	80.87	80.0
Normalized Performance	1.00	1.11	0.943	Normalized Performance	1.00	0.96	1.02
Normalized GOPS/W	1.00	0.87	0.958	Normalized GOPS/W	1.00	1.02	1.01
Memory intensive Benchmark				Computing intensive to Memory intensive benchmark			
	Baseline	HRU	Proposed		Baseline	HRU	Proposed
Peak Temperature	62.70	62.33	62.15	Peak Temperature	78.36	80.16	77.458
Normalized Performance	1.00	1.06	1.01	Normalized Performance	1.00	0.96	0.98
Normalized GOPS/W	1.00	1.36	1.00	Normalized GOPS/W	1.00	1.12	1.01
Memory & Computing Benchmark				Average Temperature Reduction : 13.6% Average Performance Overhead : 1.1% Average Energy Overhead : 1.3 %			
	Baseline	HRU	Proposed				
Peak Temperature	79.72	81.92	77.8				
Normalized Performance	1.00	1.14	1.00				
Normalized GOPS/W	1.00	0.83	0.96				

圖 5-9 模擬每組熱管理技術之峰值溫度比較

最後將前面兩個實驗的每個模擬結果整理如圖 5-9，可以看到在每個模擬的溫度部分，Proposed Design 都相對於其他兩者來的低，同時相較 Baseline 來說不會損失過多的效能與能量消耗。總結來說，本論文所提出的方法與一般熱管理的演算法相比，結果顯示，整體而言平均可以降低約 13.6% 的峰值溫度，同時只付出約 1.1% 的效能代價與 1.3% 的能量消耗。

5.3 額外硬體付出

對於本論文所提出之動態熱管理而言，由於使用到 DVFS 來調整電壓頻率，因此需要考慮到額外的 PLL、DC-DC converter，並且在現有的 Task Dispatcher Unit 上增加新的工作排程機制來配合整體演算法的運作，最後則是控制整個動態熱管理演算法的 Temperature & Power Management Unit (TPM)。在分析額外硬體付出的部分，PLL 方面參考了文獻[1]做為 PLL 功率消耗的參考，整體來說在 40nm 製程之下 PLL 本身的功率消耗大約為 2.8mW，對於整體通用計算圖形處理器而言的功率代價(power overhead)約為 0.062%；而 DC-DC converter 方面則是參考 TSMC Austin 於文獻[2]中提出的 DC-DC converter，其於 40nm 製程之下，DC-DC converter 的功率消耗約為 135 μ W，對於整體通用計算圖形處理器而言的功率代價(power overhead)約為 0.003%，而在調整電壓時需要一段時間來穩定電壓，向上調整時為 35 μ s/V(以本論文所需之調整來說大約需要 1575ns)，而向下調整則是 38 μ s/V(以本論文所需之調整大約需要 1710ns)，在運行多個測試程式之後，平均的時序代價(timing overhead)約為 0.062%；而在 TPM 與 TDU 的部分則是將演算法的行為以 Verilog 實現，於 40nm 製程下合成之功率代價約為 0.149%，對於整體通用計算圖形處理器架構而言皆為可忽略之硬體代價付出。

第 6 章 結論和未來工作

6.1 結論

本論文提出了一個考慮通用計算圖形處理器工作流程的動態熱管理方法，利用即時回饋的溫度與熱溫差作為指標來控制工作排程，並且利用時脈閘控的低功耗設計來降低高溫串流處理器，同時透過動態電壓頻率調整運算核心來消除熱溫差。在演算法的部份，根據每次的熱溫差動態調整閾值，可以避免對工作流程的過度調整而導致效能降低，並能適應更多不同的應用程式。

本論文所提出的方法與一般熱管理的演算法相比，整體而言平均可以降低約 13.6% 的峰值溫度，同時只付出約 1.1% 的效能代價與 1.3% 的能量消耗。



6.2 未來工作

本論文之未來工作可以朝以下幾點做進一步的發展或是加強。

■ 結合功率管理的動態溫度調控方法

當前所使用的動態熱管理方法在考慮控溫時能最大化減少效能的損失，但針對節能元件而言，亦須著重於功耗管理等相關議題，將功率變動的趨勢考量其中，在未來可以考慮同時將平台中的系統功率、效能與溫度等相關資訊做考量，並多加入效能限制來修正目前的演算法，讓使用者可以根據自己的需求直接管理平台上的環境，或是使用神經網路等應用程式來檢視當前演算法的效果，再利用平台上的即時資訊進一步改善演算法。

■ 低功率技術延伸

由於調整頻率與電壓無法影響靜態電流的增長，但可以透過電源閘控等技術來讓電路系統的某些部分可以長時間內保持斷電休眠，降低靜態功耗的損失，進一步提高功率與熱管理的效率。

參考文獻

- [1] Y. Hengzhou, G. Yang, and M. Zhuo, “A 40nm/65nm process adaptive low jitter phase-locked loop,” in *Proc. 2014 International Symposium on Integrated Circuits (ISIC)*, 2014, pp. 500–503.
- [2] Y. H. J. Shi, E. Soenen, and A. R. and J. Gaither, “A wide-range DC/DC converter with 2ndorder digital compensation and direct battery connection in 40nm CMOS,” in *Proc. IEEE Cust. Integr. Circuits Conf.*, 2011, pp. 1-4.
- [3] E. Fernandes, “Internet of Things Market Size, Growth | IoT Industry Report [2020-2027].”<https://www.fortunebusinessinsights.com/industry-reports/internet-of-things-iot-market-100307>.
- [4] C. -P. Chang, “An Energy-Efficient General-Purpose Computing on GPU for Edge Computing Devices Using Hardware Runtime Utilization,” M.S. thesis, Department of Electrical Engineering, National Cheng Kung University, Tainan, Taiwan, 2019.
- [5] M. S. Mohammed, A. K. Al-Dhamari, A. A. H. Ab Rahman, N. Paraman, A. A. M. Al-Kubati, and M. N. Marsono, “Temperature-aware task scheduling for dark silicon many-core system-on-chip,” in *Proc. 2019 8th International Conference on Modeling Simulation and Applied Optimization (ICMSAO)*, Manama, Bahrain, Apr. 2019.
- [6] G. Singla, G. Kaur, A. K. Unver, and U. Y. Ogras, “Predictive Dynamic Thermal and Power Management for Heterogeneous Mobile Platforms,” in *Proc. -Design, Autom. Test Eur. DATE*, Apr. 2015, vol. 2015-April, pp. 960–965.
- [7] P. -H. Yang, “Hybrid DTM for Multi-Core System using Advanced Heterogeneous Package,” M.S. thesis, Department of Electrical Engineering, National Cheng Kung University, Tainan, Taiwan, 2018.

- [8] K. -C. Hsu, "Performance Prediction Model on HSA-Compatible General-Purpose GPU System," M.S. Thesis, Department of Institute of Computer and Communication Engineer, National Cheng Kung University, Tainan, Taiwan, 2016.
- [9] V. Lakshminarayanan and N. Sriraam, "The effect of temperature on the reliability of electronic components," in *Proc. 2014 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT)*, Bangalore, 2014, pp. 1-6.
- [10] S. Kalra, "Effect of temperature dependence on performance of digital CMOS circuit technologies," in *Proc. 2013 INTERNATIONAL CONFERENCE ON SIGNAL PROCESSING AND COMMUNICATION (ICSC)*, Noida, 2013, pp. 392-395,
- [11] H. Sultan, "Is Leakage Power a Linear Function of Temperature? | DeepAI." <https://deepai.org/publication/is-leakage-power-a-linear-function-of-temperature>
- [12] D. Cuesta, J. L. Risco-Martín, J. L. Ayala, and J. I. Hidalgo, "Thermal-aware floorplanner for 3D IC, including TSVs, liquid microchannels and thermal domains optimization," *Appl. Soft Computing*, vol. 34, pp. 164–177, Sep. 2015.
- [13] S. K. Srivathsa, V. B. Suresh, P. Panchapakeshan, and S. Kundu, "Dynamic thermal management for system-on-chip using bus arbitration," in *Proc. 2010 International SoC Design Conference*. Seoul, 2010, pp. 372-375,
- [14] A. Kumar, L. Shang, L. S. Peh, and N. K. Jha, "System-Level Dynamic Thermal Management for High-Performance Microprocessors," *IEEE Trans. Comput. Des. Integr. Circuits Syst.*, vol. 27, no. 1, pp. 96–107, Jan. 2008.
- [15] B. Salami, M. Baharani, and H. Noori, "An adaptive temperature threshold schema for dynamic thermal management of multi-core processors," in *Proc. - 17th CSI Int. Symp. Comput. Archit. Digit. Syst. CADs 2013*, 2013, pp. 119–120.

- [16] W. -S. Hsieh, "Design of Cycle-accurate SIMT Core and Implementation," M.S. Thesis, Department of Institute of Computer and Communication Engineer, National Cheng Kung University, Tainan, Taiwan, Tainan, Taiwan, 2016.
- [17] J. -H. Jheng, "Micro-Architecture Optimization of HSA-Compatible GPU," M.S. thesis, Department of Institute of Computer and Communication Engineer, National Cheng Kung University, Tainan, Taiwan, Tainan, Taiwan, 2018.
- [18] L. Liu and D. Qi, "An Independent Task Scheduling Algorithm in Heterogeneous Multi-core Processor Environment," in *Proc. 2018 IEEE 3rd Adv. Inf. Technol. Electron. Autom. Control Conf. IAEAC 2018*, Dec. 2018, pp. 142–146.
- [19] G. Wu, Z. Xu, Q. Xia, J. Ren and F. Xia, "Task Allocation and Migration Algorithm for Temperature-Constrained Real-Time Multi-Core Systems," in *Proc. 2010 IEEE/ACM Int'l Conference on Green Computing and Communications & Int'l Conference on Cyber, Physical and Social Computing*, Hangzhou, 2010, pp. 189-196.
- [20] J. Cha, J. Kim, and Y. Park, "Core-level DVFS for Spatial Multitasking GPUs," in *Proc. IEEE Reg. 10 Annu. Int. Conf. (TENCON)*, Feb. 2019, vol. 2018-October, pp. 1525–1528.
- [21] M. H. Vo, "The Merged Clock Gating Architecture for Low Power Digital Clock Application on FPGA," in *Proc. Int. Conf. Adv. Technol. Commun.*, Dec. 2018, vol. 2018-October, pp. 282–286.
- [22] G. Bhargave, S. Uniyal, and P. Sheokand, "Low Power Adiabatic 4-Bit Johnson Counter Based on Power-Gating Cpl Logic," in *Proc. 2nd IEEE Int. Conf. Innov. Appl. Comput. Intell. Power, Energy Control. with their Impact Humanit. CIPECH 2016*, May 2017, pp. 297–301.

- [23] B. Yu, S. Dong, and S. Goto, "Multi-Voltage and Level-Shifter Assignment Driven Floorplanning," in *ASICON 2009 - Proc. 2009 8th IEEE Int. Conf. ASIC*, 2009, pp. 1264–1267.
- [24] "JEDEC STANDARD." http://www.softnology.biz/pdf/JESD79-4_DDR4_SDRAM.pdf?fbclid=IwAR1doOEVrs8Y6UK3JsNjaiUs9gvu1OhXSxNVUjVbKFAC8kxROPRAAN8T58A (accessed Jul. 15, 2020).
- [25] W. -X. YANG, "Fast and Effective Thermal Analysis for Integrated Circuits Using 3-Dimensional Discontinuous Galerkin Method," M.S. thesis, Department of Electrical Engineering, National Cheng Kung University, Tainan, Taiwan, 2019.

